

Capítulo 2

Listas Basadas en Arreglos

En este capítulo, estudiaremos implementaciones de las interfaces `List` y `Queue` donde los datos subyacentes se almacenan en un arreglo, llamado arreglo de respaldo (*backing array*). La siguiente tabla resume los tiempos de ejecución de las operaciones para las estructuras de datos presentadas en este capítulo:

	<code>get(i)/set(i,x)</code>	<code>add(i,x)/remove(i)</code>
<code>ArrayStack</code>	$O(1)$	$O(n-1)$
<code>ArrayDeque</code>	$O(1)$	$O(\min\{i, n-i\})$
<code>DualArrayDeque</code>	$O(1)$	$O(\min\{i, n-i\})$
<code>RootishArrayStack</code>	$O(1)$	$O(n-i)$

Las estructuras de datos que funcionan almacenando datos en un solo arreglo tienen muchas ventajas y limitaciones en común:

- Los arreglos ofrecen acceso en tiempo constante a cualquier valor en el arreglo. Esto es lo que permite que `get(i)` y `set(i, x)` se ejecuten en tiempo constante.
- Los arreglos no son muy dinámicos. Agregar o eliminar un elemento cerca de la mitad de una lista significa que un gran número de elementos en el arreglo deben desplazarse para hacer espacio para el elemento recién agregado o para llenar el vacío creado por el elemento eliminado. Es por esto que las operaciones `add(i, x)` y `remove(i)` tienen tiempos de ejecución que dependen de n y de i .
- Los arreglos no pueden expandirse ni contraerse. Cuando el número de elementos en la estructura de datos excede el tamaño del arreglo de respaldo, se debe asignar un nuevo arreglo y los datos del arreglo antiguo deben copiarse en el nuevo arreglo. Esta es una operación costosa.

El tercer punto es importante. Los tiempos de ejecución citados en la tabla anterior no incluyen el costo asociado con el crecimiento y la contracción del arreglo de respaldo. Veremos que, si se gestiona cuidadosamente, el costo de hacer crecer y encoger el arreglo de respaldo no añade mucho al costo de una *operación promedio*. Más precisamente, si comenzamos con una estructura de datos vacía, y realizamos cualquier secuencia de m operaciones `add(i,x)` o `remove(i)`, entonces el costo total de hacer crecer y encoger el arreglo de respaldo, sobre la secuencia completa de m operaciones es $O(m)$. Aunque algunas operaciones individuales son más costosas, el costo amortizado, cuando se amortiza sobre todas las m operaciones, es solo $O(1)$ por operación.

En este capítulo, y a lo largo de este libro, será conveniente tener arreglos que hagan un seguimiento de su tamaño. Los arreglos habituales de C++ no hacen esto, por lo que hemos definido una clase, `array`, que hace un seguimiento de su longitud. La implementación de esta clase es sencilla. Se implementa como un arreglo estándar de C++, `a`, y un entero, `length`:

array

```
T *a;
int length;
```

El tamaño de un **array** se especifica en el momento de la creación:

array

```
array(int len) {
    length = len;
    a = new T[length];
}
```

Se puede acceder a los elementos de un arreglo mediante índices:

Los elementos de un arreglo pueden indexarse:

array

```
T& operator[](int i) {
    assert(i >= 0 && i < length);
    return a[i];
}
```

Por último, cuando se asigna un array a otro, se trata simplemente de una manipulación de punteros que requiere un tiempo constante:

ArrayStack: Operaciones Rápidas de Pila Usando un Arreglo

§2.1

array

```
array<T>& operator=(array<T> &b) {
    if (a != NULL) delete[] a;
    a = b.a;
    b.a = NULL;
    length = b.length;
    return *this;
}
```

2.1 ArrayStack: Operaciones Rápidas de Pila Usando un Arreglo

Un **ArrayStack** implementa la interfaz de lista utilizando un arreglo **a**, llamado el *arreglo de respaldo*. El elemento de la lista con índice **i** se almacena en **a[i]**. La mayoría de las veces, **a** es más grande de lo estrictamente necesario, por lo que se utiliza un entero **n** para llevar un registro del número de elementos

realmente almacenados en `a`. De esta manera, los elementos de la lista se almacenan en `a[0]`, ..., `a[n-1]` y, en todo momento, `a.length >= n`.

ArrayStack

```
array<T> a;
int n;
int size() {
    return n;
}
```

2.1.1 Los Fundamentos

Acceder y modificar los elementos de un `ArrayStack` usando `get(i)` y `set(i,x)` es trivial. Después de realizar las comprobaciones de límites necesarias, simplemente devolvemos o establecemos, respectivamente, `a[i]`.

ArrayStack

```
T get(int i) {
    return a[i];
}
T set(int i, T x) {
    T y = a[i];
    a[i] = x;
    return y;
}
```

Las operaciones de añadir y eliminar elementos de un `ArrayStack` se ilustran en la Figura 2.1. Para implementar la operación `add(i, x)`, primero comprobamos si `a` está ya lleno. Si es así, llamamos al método `resize()` para incrementar el tamaño de `a`. Cómo se implementa `resize()` se discutirá más adelante. Por ahora, es suficiente saber que, tras una llamada a `resize()`, podemos estar seguros de que `a.length > n`. Una vez solucionado esto, ahora desplazamos los elementos `a[i], ..., a[n-1]` una posición a la derecha para hacer sitio para `x`, establecemos `a[i]` igual a `x`, e incrementamos `n`.

ArrayStack

```
void add(int i, T x) {
    if (n + 1 > a.length) resize();
    for (int j = n; j > i; j--)
        a[j] = a[j - 1];
    a[i] = x;
    n++;
}
```

Si ignoramos el costo de la posible llamada a `resize()`, entonces el costo de la operación `add(i,x)` es proporcional al número de elementos que tenemos que desplazar para hacer espacio para `x`. Por lo tanto, el costo de esta operación (ignorando el costo de redimensionar `a`) es $O(n - i)$.

Implementar la operación `remove(i)` es similar. Desplazamos los elementos `a[i+1], ..., a[n-1]` una posición a la izquierda (sobrescribiendo `a[i]`) y decrementamos el valor de `n`. Después de hacer esto, comprobamos si `n` se está volviendo mucho más pequeño que `a.length` verificando si `a.length \ge 3n`. Si es así, entonces llamamos a `resize()` para reducir el tamaño de `a`.

ArrayStack

```
T remove(int i) {
    T x = a[i];
    for (int j = i; j < n - 1; j++)
        a[j] = a[j + 1];
    n--;
    if (a.length >= 3 * n) resize();
}
```

IMG

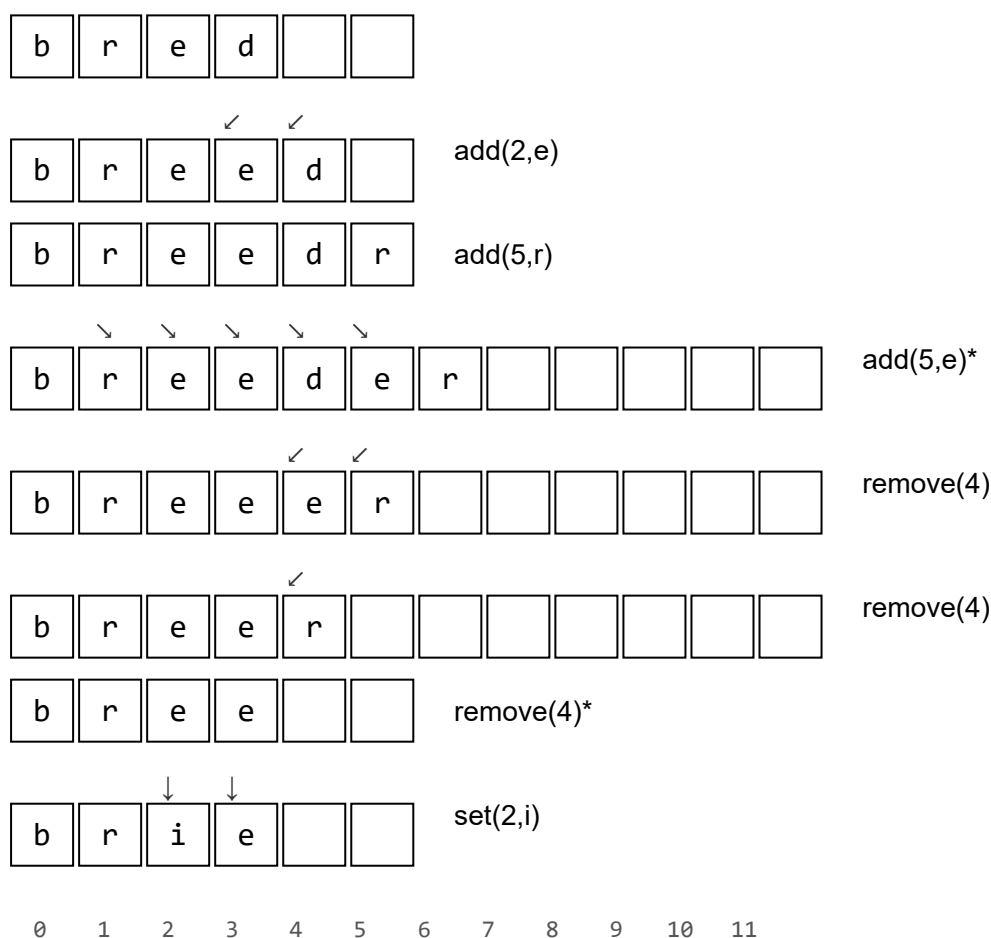


Figura 2.1: Una secuencia de operaciones `add(i,x)` y `remove(i)` en un `ArrayStack`. Las flechas denotan elementos que están siendo copiados. Las operaciones que resultan en una llamada a `resize()` están marcadas con un asterisco.

```
    return x;
}
```

Si ignoramos el costo del método `resize()`, el costo de una operación `remove(i)` es proporcional al número de elementos que desplazamos, que es $O(n - i)$.

2.1.2 Crecimiento y Contracción

El método `resize()` es bastante sencillo; asigna un nuevo arreglo `b` cuyo tamaño es $2n$ y copia los n elementos de `a` en las primeras n posiciones de `b`, y luego establece `a` en `b`. Por lo tanto, después de una llamada a `resize()`, `a.length = 2n`.

ArrayStack

```
void resize() {
    array<T> b(max(2 * n, 1));
    for (int i = 0; i < n; i++)
        b[i] = a[i];
    a = b;
}
```

Analizar el costo real de la operación `resize()` es fácil. Asigna un arreglo `b` de tamaño $2n$ y copia los n elementos de `a` en `b`. Esto toma un tiempo $O(n)$.

El análisis del tiempo de ejecución de la sección anterior ignoró el costo de las llamadas a `resize()`. En esta sección analizamos este costo utilizando una técnica conocida como análisis amortizado. Esta técnica no intenta determinar el costo de redimensionar durante cada operación individual `add(i, x)` y `remove(i)`. En su lugar, considera el costo de todas las llamadas a `resize()` durante una secuencia de m llamadas a `add(i, x)` o `remove(i)`. En particular, mostraremos:

Lema 2.1. Si se crea un `ArrayStack` vacío y se realiza cualquier secuencia de $m \geq 1$ llamadas a `add(i, x)` y `remove(i)`, entonces el tiempo total empleado durante todas las llamadas a `resize()` es $O(m)$.

Prueba. Demostraremos que cada vez que se llama a `resize()`, el número de llamadas a `add` o `remove` desde la última llamada a `resize()` es al menos $n/2 - 1$. Por lo tanto, si n_i denota el valor de n durante la i -ésima llamada a `resize()` y r denota el número de llamadas a `resize()`, entonces el número total de llamadas a `add(i, x)` o `remove(i)` es al menos

$$\sum_{i=1}^r (n_i/2 - 1) \leq m,$$

lo cual es equivalente a

$$\sum_{i=1}^r n_i \leq 2m + 2r.$$

Por otro lado, el tiempo total empleado durante todas las llamadas a `resize()` es

$$\sum_{i=1}^r O(n_i) \leq O(m + r) = O(m),$$

dado que r no es mayor que m . Todo lo que queda es demostrar que el número de llamadas a `add(i,x)` o `remove(i)` entre la $(i - 1)$ -ésima y la i -ésima llamada a `resize()` es al menos $n_i/2$.

Hay dos casos a considerar. En el primer caso, `resize()` está siendo llamado por `add(i,x)` porque el arreglo de respaldo `a` está lleno, es decir, `a.length = n = n_i`. Consideremos la llamada anterior a `resize()`: después de esta llamada anterior, el tamaño de `a` era `a.length`, pero el número de elementos almacenados en `a` era como máximo `a.length/2 = n_i/2`. Pero ahora el número de elementos almacenados en `a` es `n_i = a.length`, por lo que debe haber habido al menos $n_i/2$ llamadas a `add(i,x)` desde la llamada anterior a `resize()`.

El segundo caso ocurre cuando `remove(i)` llama a `resize()` porque `a.length >= 3n = 3n_i`. De nuevo, después de la llamada anterior a `resize()` el número de elementos almacenados en `a` era al menos $a.length/2 - 1$.¹ Ahora hay $n_i \leq a.length/3$ elementos almacenados en `a`. Por lo tanto, el número de operaciones `remove(i)` desde la última llamada a `resize()` es al menos

$$\begin{aligned} R &\geq a.length/2 - 1 - a.length/3 \\ &= a.length/6 - 1 \\ &= (a.length/3)/2 - 1 \\ &\geq n_i/2 - 1. \end{aligned}$$

En cualquier caso, el número de llamadas a `add(i,x)` o `remove(i)` que ocurren entre la $(i - 1)$ -ésima llamada a `resize()` y la i -ésima llamada a `resize()` es al menos $n_i/2 - 1$, como se requiere para completar la prueba. $\square$

¹El -1 en esta fórmula tiene en cuenta el caso especial que ocurre cuando $n = 0$ y `a.length = 1`.

2.1.3 Resumen

El siguiente teorema resume el rendimiento de un `ArrayStack`:

Teorema 2.1. Un `ArrayStack` implementa la interfaz `List`. Ignorando el costo de las llamadas a `resize()`, un `ArrayStack` soporta las operaciones

- `get(i)` y `set(i,x)` en tiempo $O(1)$ por operación; y
- `add(i,x)` y `remove(i)` en tiempo $O(1 + n - i)$ por operación.

Además, comenzando con un `ArrayStack` vacío y realizando cualquier secuencia de m operaciones `add(i,x)` y `remove(i)` resulta en un total de $O(m)$ tiempo empleado durante todas las llamadas a `resize()`.

El `ArrayStack` es una forma eficiente de implementar una Pila (*Stack*). En particular, podemos implementar `push(x)` como `add(n,x)` y `pop()` como `remove(n - 1)`, en cuyo caso estas operaciones se ejecutarán en tiempo amortizado $O(1)$.

2.2 FastArrayStack: Un ArrayStack Optimizado

Gran parte del trabajo realizado por un `ArrayStack` implica desplazar (mediante `add(i, x)` y `remove(i)`) y copiar (mediante `resize()`) datos. En las implementaciones mostradas anteriormente, esto se hacía usando bucles `for`. Resulta que muchos entornos de programación tienen funciones específicas que son muy eficientes para copiar y mover bloques de datos. En el lenguaje de programación C, están las funciones `memcpy(d, s, n)` y `memmove(d, s, n)`. En el lenguaje C++ está el algoritmo `std::copy(a0, a1, b)`. En Java está el método `System.arraycopy(s, i, d, j, n)`.

FastArrayStack

```
void resize() {
    array<T> b(max(1, 2*n));
    std::copy(a+0, a+n, b+0);
    a = b;
}

void add(int i, T x) {
    if (n + 1 > a.length) resize();
    std::copy_backward(a+i, a+n, a+n+1);
    a[i] = x;
    n++;
}
```

Estas funciones suelen estar altamente optimizadas e incluso pueden usar instrucciones de máquina especiales que pueden hacer esta copia mucho más rápido de lo que podríamos hacerlo usando un bucle `for`. Aunque el uso de estas funciones no disminuye asintóticamente los tiempos de ejecución, aún puede ser una optimización que vale la pena.

En las implementaciones de C++, el uso del nativo `std::copy(a0, a1, b)` resultó en aceleraciones de un factor entre 2 y 3, dependiendo de los tipos de operaciones realizadas. Su experiencia puede variar.

2.3 ArrayQueue: Una Cola Basada en Arreglos

En esta sección, presentamos la estructura de datos `ArrayQueue`, que implementa una cola FIFO (primero en entrar, primero en salir); los elementos se eliminan (usando la operación `remove()`) de la cola en el mismo orden en que se agregan (usando la operación `add()`).

Observe que un `ArrayStack` es una mala elección para la implementación de una cola FIFO. No es una buena elección porque debemos elegir un extremo de la lista en el cual agregar elementos y luego eliminar elementos del otro extremo. Una de las dos operaciones trabajará en la cabeza de la lista, lo que implica llamar a `add(i, x)` o `remove(i)` con un valor de $i = 0$. Esto da un tiempo de ejecución proporcional a n .

Para obtener una implementación eficiente de una cola basada en arreglos, primero notamos que el problema sería fácil si tuviéramos un arreglo infinito `a`. Podríamos mantener un índice `j` que realice un seguimiento del siguiente elemento a eliminar y un entero `n` que cuente el número de elementos en la cola. Los elementos de la cola siempre se almacenarían en

$$a[j], a[j + 1], \dots, a[j + n - 1].$$

Inicialmente, tanto `j` como `n` se establecerían en 0. Para agregar un elemento, lo colocaríamos en `a[j + n]` e incrementaríamos `n`. Para eliminar un elemento, lo eliminaríamos de `a[j]`, incrementaríamos `j` y

decrementaríamos n .

Por supuesto, el problema con esta solución es que requiere un arreglo infinito. Un `ArrayQueue` simula esto utilizando un arreglo finito `a` y *aritmética modular*. Este es el tipo de aritmética que se utiliza cuando hablamos de la hora del día. Por ejemplo, las 10:00 más cinco horas dan las 3:00. Formalmente, decimos que

$$10 + 5 = 15 \equiv 3 \pmod{12}.$$

Leemos la última parte de esta ecuación como "15 es congruente con 3 módulo 12". También podemos tratar `mod` como un operador binario, de modo que

$$15 \bmod 12 = 3.$$

De manera más general, para un entero a y un entero positivo m , $a \bmod m$ es el único entero $r \in \{0, \dots, m - 1\}$ tal que $a = r + km$ para algún entero k . De manera menos formal, el valor r es el resto que obtenemos cuando dividimos a entre m . En muchos lenguajes de programación, incluido C++, el operador mod se representa usando el símbolo `%`.²

La aritmética modular es útil para simular un arreglo infinito, ya que $i \bmod a.length$ siempre da un valor en el rango $0, \dots, a.length - 1$. Usando aritmética modular podemos almacenar los elementos de la cola en las ubicaciones del arreglo

$$a[j \% a.length], a[(j + 1) \% a.length], \dots, a[(j + n - 1) \% a.length].$$

Esto trata al arreglo `a` como un *arreglo circular* en el cual los índices mayores que `a.length - 1` "dan la vuelta" al principio del arreglo.

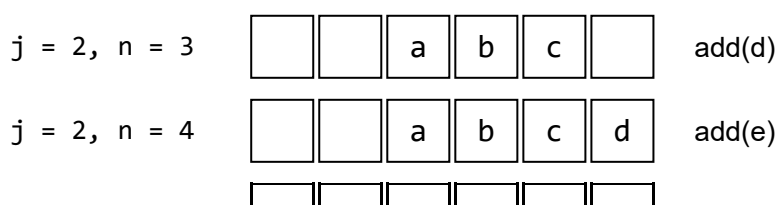
Lo único que queda por preocuparnos es asegurarnos de que el número de elementos en el `ArrayQueue` no exceda el tamaño de `a`.

ArrayQueue

```
array<T> a;
int j;
int n;
```

Una secuencia de operaciones `add(x)` y `remove()` en un `ArrayQueue` se ilustra en la Figura 2.2. Para implementar `add(x)`, primero verificamos si `a` está lleno y, si es necesario, llamamos a `resize()` para incrementar el tamaño de `a`. A continuación, almacenamos `x` en `a[(j + n) % a.length]` e incrementamos `n`.

²Esto a veces se conoce como el operador mod "descerebrado", ya que no implementa correctamente el operador mod matemático cuando el primer argumento es negativo.



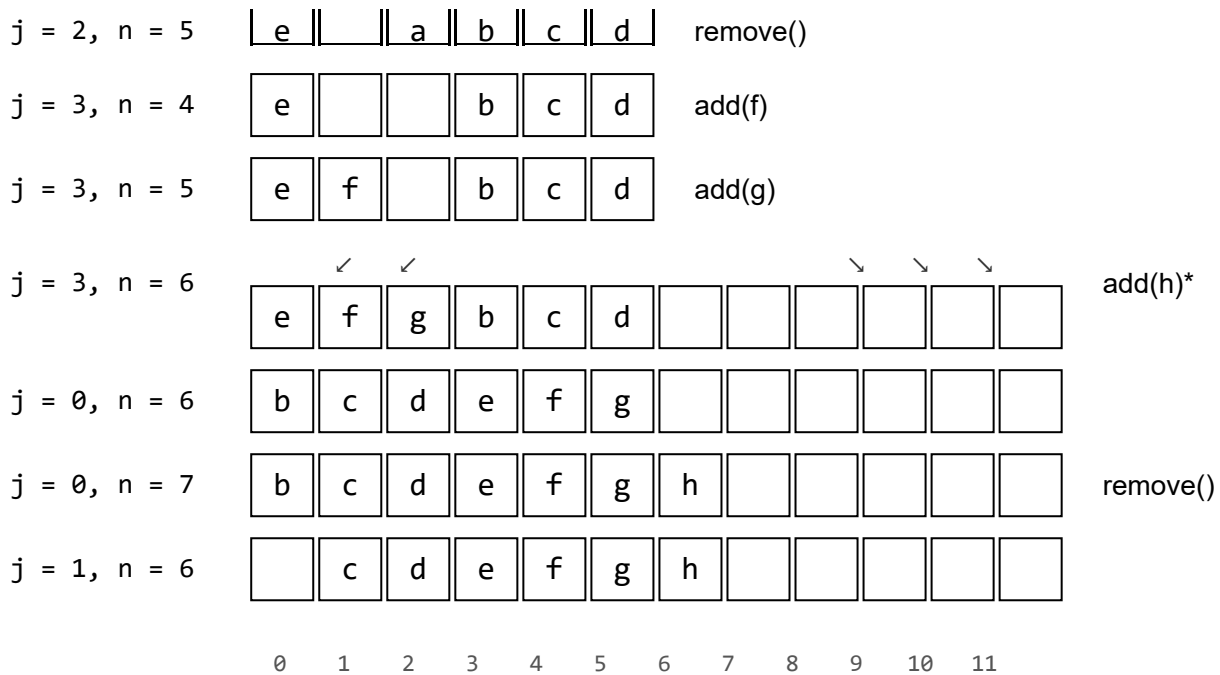


Figura 2.2: Una secuencia de operaciones `add(x)` y `remove(i)` en un `ArrayQueue`. Las flechas denotan elementos que están siendo copiados. Las operaciones que resultan en una llamada a `resize()` están marcadas con un asterisco.

ArrayQueue

```
bool add(T x) {
    if (n + 1 > a.length) resize();
    a[(j+n) % a.length] = x;
    n++;
    return true;
}
```

Para implementar `remove()`, primero almacenamos `a[j]` para poder devolverlo más tarde. A continuación, decrementamos `n` e incrementamos `j` (módulo `a.length`) estableciendo `j = (j + 1) mod a.length`. Finalmente, devolvemos el valor almacenado de `a[j]`. Si es necesario, podemos llamar a `resize()` para disminuir el tamaño de `a`.

ArrayQueue

```
T remove() {
    T x = a[j];
    j = (j + 1) % a.length;
    n--;
    if (a.length >= 3*n) resize();
    return x;
}
```

Finalmente, la operación `resize()` es muy similar a la operación `resize()` de `ArrayStack`. Asigna un nuevo arreglo, `b`, de tamaño `2n` y copia

$$a[j], a[(j + 1) \% a.length], \dots, a[(j + n - 1) \% a.length]$$

en

$$b[0], b[1], \dots, b[n - 1]$$

y establece $j = 0$.

ArrayQueue

```
void resize() {
    array<T> b(max(1, 2*n));
    for (int k = 0; k < n; k++)
        b[k] = a[(j+k)%a.length];
    a = b;
    j = 0;
}
```

2.3.1 Resumen

El siguiente teorema resume el rendimiento de la estructura de datos **ArrayQueue**:

Teorema 2.2. Un **ArrayQueue** implementa la interfaz **Queue** (FIFO). Ignorando el costo de las llamadas a **resize()**, un **ArrayQueue** soporta las operaciones **add(x)** y **remove()** en tiempo $O(1)$ por operación. Además, comenzando con un **ArrayQueue** vacío, cualquier secuencia de m operaciones **add(i, x)** y **remove(i)** resulta en un total de $O(m)$ tiempo empleado durante todas las llamadas a **resize()**.

2.4 ArrayDeque: Operaciones Rápidas de Deque Usando un Arreglo

El **ArrayQueue** de la sección anterior es una estructura de datos para representar una secuencia que nos permite agregar eficientemente a un extremo de la secuencia y eliminar del otro extremo. La estructura de datos **ArrayDeque** permite la adición y eliminación eficientes en ambos extremos. Esta estructura implementa la interfaz **List** utilizando la misma técnica de arreglo circular utilizada para representar un **ArrayQueue**.

ArrayDeque

```
array<T> a;
int j;
int n;
```

Las operaciones **get(i)** y **set(i, x)** en un **ArrayDeque** son sencillas. Obtienen o establecen el elemento del arreglo $a[(j + i) \% a.length]$.

ArrayDeque

```
T get(int i) {
    return a[(j + i) % a.length];
}
```

```

T set(int i, T x) {
    T y = a[(j + i) % a.length];
    a[(j + i) % a.length] = x;
    return y;
}

```

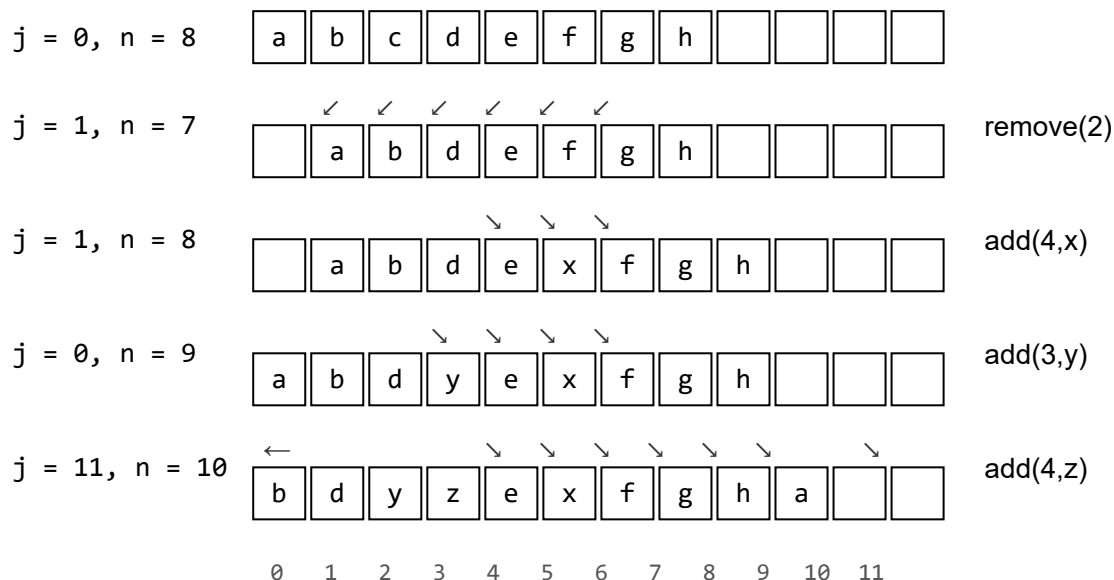


Figura 2.3: Una secuencia de operaciones `add(i, x)` y `remove(i)` en un `ArrayDeque`. Las flechas denotan elementos que están siendo copiados.

La implementación de `add(i, x)` es un poco más interesante. Como de costumbre, primero verificamos si `a` está lleno y, si es necesario, llamamos a `resize()` para redimensionar `a`. Recordemos que queremos que esta operación sea rápida cuando `i` es pequeño (cerca de 0) o cuando `i` es grande (cerca de `n`). Por lo tanto, verificamos si $i < n/2$. Si es así, desplazamos los elementos `a[0], ..., a[i-1]` una posición a la izquierda. De lo contrario ($i \geq n/2$), desplazamos los elementos `a[i], ..., a[n-1]` una posición a la derecha. Véase la Figura 2.3 para una ilustración de las operaciones `add(i, x)` y `remove(x)` en un `ArrayDeque`.

ArrayDeque

```

void add(int i, T x) {
    if (n + 1 > a.length) resize();
    if (i < n/2) { // desplazar a[0], ..., a[i-1] una posición a la izquierda
        j = (j == 0) ? a.length - 1 : j - 1;
        for (int k = 0; k <= i-1; k++)
            a[(j+k)%a.length] = a[(j+k+1)%a.length];
    } else { // desplazar a[i], ..., a[n-1] una posición a la derecha
        for (int k = n; k > i; k--)
            a[(j+k)%a.length] = a[(j+k-1)%a.length];
    }
    a[(j+i)%a.length] = x;
    n++;
}

```

Al realizar el desplazamiento de esta manera, garantizamos que `add(i, x)` nunca tenga que desplazar más de $\min\{i, n - i\}$ elementos. Por lo tanto, el tiempo de ejecución de la operación `add(i, x)` (ignorando el costo de una operación `resize()`) es $O(1 + \min\{i, n - i\})$.

La implementación de la operación `remove(i)` es similar. Desplaza los elementos `a[0], ..., a[i-1]` una posición a la derecha o desplaza los elementos `a[i+1], ..., a[n-1]` una posición a la izquierda dependiendo de si $i < n/2$. De nuevo, esto significa que `remove(i)` nunca emplea más de $O(1 + \min\{i, n - i\})$ tiempo en desplazar elementos.

ArrayDeque

```
T remove(int i) {
    T x = a[(j+i)%a.length];
    if (i < n/2) { // desplazar a[0],...,a[i-1] una posición a la derecha
        for (int k = i; k > 0; k--)
            a[(j+k)%a.length] = a[(j+k-1)%a.length];
        j = (j + 1) % a.length;
    } else { // desplazar a[i+1],...,a[n-1] una posición a la izquierda
        for (int k = i; k < n-1; k++)
            a[(j+k)%a.length] = a[(j+k+1)%a.length];
    }
    n--;
    if (3*n < a.length) resize();
    return x;
}
```

2.4.1 Resumen

El siguiente teorema resume el rendimiento de la estructura de datos `ArrayDeque`:

Teorema 2.3. *Un `ArrayDeque` implementa la interfaz `List`. Ignorando el costo de las llamadas a `resize()`, un `ArrayDeque` soporta las operaciones*

- `get(i)` y `set(i, x)` en tiempo $O(1)$ por operación; y
- `add(i, x)` y `remove(i)` en tiempo $O(1 + \min\{i, n - i\})$ por operación.

Además, comenzando con un `ArrayDeque` vacío, realizar cualquier secuencia de m operaciones `add(i, x)` y `remove(i)` resulta en un total de $O(m)$ tiempo empleado durante todas las llamadas a `resize()`.

2.5 DualArrayDeque: Construyendo un Deque a partir de Dos Pilas

A continuación, presentamos una estructura de datos, el `DualArrayDeque`, que logra los mismos límites de rendimiento que un `ArrayDeque` utilizando dos `ArrayStack`. Aunque el rendimiento asintótico del `DualArrayDeque` no es mejor que el del `ArrayDeque`, todavía vale la pena estudiarlo, ya que ofrece un buen ejemplo de cómo construir una estructura de datos sofisticada combinando dos estructuras de datos más simples.

Un `DualArrayDeque` representa una lista utilizando dos `ArrayStack`. Recordemos que un `ArrayStack` es rápido cuando las operaciones sobre él modifican elementos cerca del final. Un `DualArrayDeque` coloca dos

ArrayStack, llamados **front** y **back**, uno tras otro (espalda con espalda) para que las operaciones sean rápidas en cualquiera de los dos extremos.

DualArrayDeque

```
ArrayStack<T> front;
ArrayStack<T> back;
```

Un **DualArrayDeque** no almacena explícitamente el número, n , de elementos que contiene. No necesita hacerlo, ya que contiene $n = \text{front.size()} + \text{back.size()}$ elementos. Sin embargo, al analizar el **DualArrayDeque** seguiremos utilizando n para denotar el número de elementos que contiene.

DualArrayDeque

```
int size() {
    return front.size() + back.size();
}
```

El **ArrayStack front** almacena los elementos de la lista cuyos índices son $0, \dots, \text{front.size()} - 1$, pero los almacena en orden inverso. El **ArrayStack back** contiene elementos de la lista con índices en $\text{front.size()}, \dots, \text{size()} - 1$ en el orden normal. De esta manera, $\text{get}(i)$ y $\text{set}(i, x)$ se traducen en llamadas apropiadas a $\text{get}(i)$ o $\text{set}(i, x)$ ya sea en **front** o en **back**, las cuales toman un tiempo $O(1)$ por operación.

DualArrayDeque

```
T get(int i) {
    if (i < front.size()) {
        return front.get(front.size() - i - 1);
    } else {
        return back.get(i - front.size());
    }
}

T set(int i, T x) {
    if (i < front.size()) {
        return front.set(front.size() - i - 1, x);
    } else {
        return back.set(i - front.size(), x);
    }
}
```

Observe que si un índice $i < \text{front.size()}$, entonces corresponde al elemento de **front** en la posición $\text{front.size()} - i - 1$, ya que los elementos de **front** se almacenan en orden inverso.

Agregar y eliminar elementos de un **DualArrayDeque** se ilustra en la Figura 2.4. La operación $\text{add}(i, x)$ manipula ya sea **front** o **back**, según corresponda:

DualArrayDeque

```
void add(int i, T x) {
    if (i < front.size()) {
        front.add(front.size() - i, x);
    } else {
        back.add(i - front.size(), x);
    }
    balance();
}
```

El método `add(i, x)` realiza un reequilibrio de los dos `ArrayStacks` `front` y `back`, llamando al método `balance()`. La implementación de `balance()` se describe a continuación, pero por ahora es suficiente saber que `balance()` asegura que, a menos que `size() < 2`, `front.size()` y `back.size()` no difieran en más de un factor de 3. En particular, $3 \cdot \text{front.size()} \geq \text{back.size()}$ y $3 \cdot \text{back.size()} \geq \text{front.size()}$.

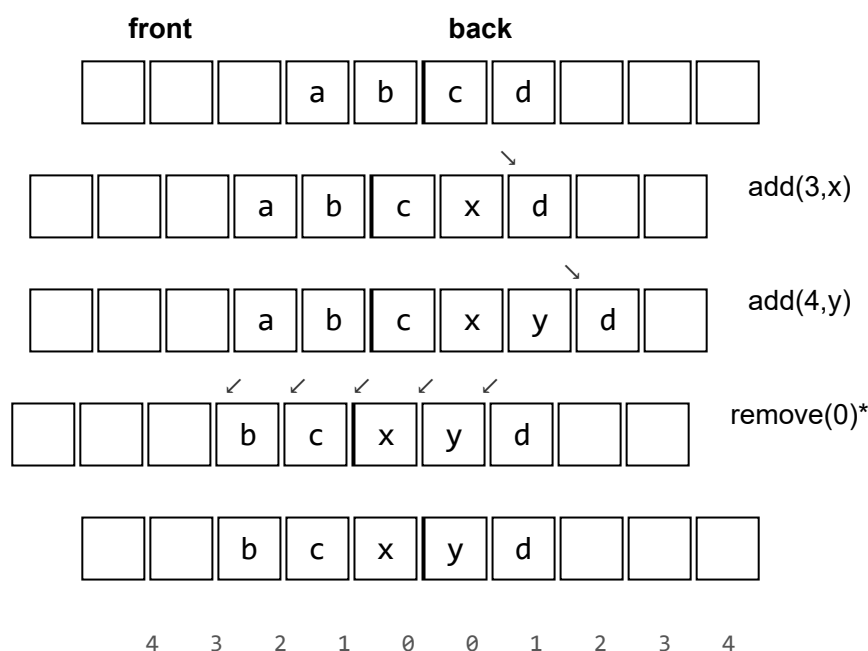


Figura 2.4: Una secuencia de operaciones `add(i, x)` y `remove(i)` en un `DualArrayDeque`. Las flechas denotan elementos que están siendo copiados. Las operaciones que resultan en un reequilibrio mediante `balance()` están marcadas con un asterisco.

A continuación analizamos el costo de `add(i, x)`, ignorando el costo de las llamadas a `balance()`. Si $i < \text{front.size()}$, entonces `add(i, x)` se implementa mediante la llamada a `front.add(front.size() - i - 1, x)`. Dado que `front` es un `ArrayStack`, el costo de esto es

$$O(\text{front.size()} - (\text{front.size()} - i - 1) + 1) = O(i + 1). \quad (2.1)$$

Por otro lado, si $i \geq \text{front.size()}$, entonces `add(i, x)` se implementa como `back.add(i - front.size(), x)`. El costo de esto es

$$O(\text{back.size()} - (i - \text{front.size()}) + 1) = O(n - i + 1). \quad (2.2)$$

Observe que el primer caso (2.1) ocurre cuando $i < n/4$. El segundo caso (2.2) ocurre cuando $i \geq 3n/4$. Cuando $n/4 \leq i < 3n/4$, no podemos estar seguros de si la operación afecta a **front** o a **back**, pero en cualquier caso, la operación toma $O(n) = O(i) = O(n - i)$ tiempo, ya que $i \geq n/4$ y $n - i \geq n/4$. Resumiendo la situación, tenemos

$$\text{Tiempo de ejecución de } \text{add}(i, x) \leq \begin{cases} O(1 + i) & \text{si } i < n/4 \\ O(n) & \text{si } n/4 \leq i < 3n/4 \\ O(1 + n - i) & \text{si } i \geq 3n/4 \end{cases}$$

Por lo tanto, el tiempo de ejecución de **add**(i,x), si ignoramos el costo de la llamada a **balance**(), es $O(1 + \min\{i, n - i\})$.

La operación **remove**(i) y su análisis se asemejan a la operación **add**(i,x) y su análisis.

DualArrayDeque

```
T remove(int i) {
    T x;
    if (i < front.size()) {
        x = front.remove(front.size() - i - 1);
    } else {
        x = back.remove(i - front.size());
    }
    balance();
    return x;
}
```

2.5.1 Equilibrio

Finalmente, nos centramos en la operación **balance**() realizada por **add**(i,x) y **remove**(i). Esta operación asegura que ni **front** ni **back** se vuelvan demasiado grandes (o demasiado pequeños). Asegura que, a menos que haya menos de dos elementos, tanto **front** como **back** contengan al menos $n/4$ elementos. Si este no es el caso, entonces mueve elementos entre ellos para que **front** y **back** contengan exactamente $\lfloor n/2 \rfloor$ elementos y $\lceil n/2 \rceil$ elementos, respectivamente.

DualArrayDeque

```
void balance() {
    if (3*front.size() < back.size()
        || 3*back.size() < front.size()) {
        int n = front.size() + back.size();
        int nf = n/2;
        array<T> af(max(2*nf, 1));
        for (int i = 0; i < nf; i++)
            af[nf-i-1] = get(i);
        int nb = n - nf;
        array<T> ab(max(2*nb, 1));
        for (int i = 0; i < nb; i++)
            ab[i] = get(nf+i);
    }
}
```

```

        front.a = af;
        front.n = nf;
        back.a = ab;
        back.n = nb;
    }
}

```

Aquí hay poco que analizar. Si la operación `balance()` realiza un reequilibrio, entonces mueve $O(n)$ elementos y esto toma $O(n)$ tiempo. Esto es malo, ya que `balance()` se llama con cada llamada a `add(i, x)` y `remove(i)`. Sin embargo, el siguiente lema muestra que, en promedio, `balance()` solo emplea una cantidad constante de tiempo por operación.

Lema 2.2. Si se crea un `DualArrayDeque` vacío y se realiza cualquier secuencia de $m \geq 1$ llamadas a `add(i, x)` y `remove(i)`, entonces el tiempo total empleado durante todas las llamadas a `balance()` es $O(m)$.

Prueba. Demostraremos que, si `balance()` se ve forzado a desplazar elementos, entonces el número de operaciones `add(i, x)` y `remove(i)` desde la última vez que `balance()` desplazó elementos es al menos $n/2 - 1$. Al igual que en la prueba del Lema 2.1, esto es suficiente para demostrar que el tiempo total empleado por `balance()` es $O(m)$.

Realizaremos nuestro análisis utilizando una técnica conocida como el método del potencial. Definimos el potencial, Φ , del `DualArrayDeque` como la diferencia de tamaño entre `front` y `back`:

$$\Phi = |\text{front.size()} - \text{back.size()}|.$$

Lo interesante de este potencial es que una llamada a `add(i, x)` o `remove(i)` que no realiza ningún equilibrio puede aumentar el potencial como máximo en 1.

Observe que, inmediatamente después de una llamada a `balance()` que desplaza elementos, el potencial, Φ_0 , es como máximo 1, ya que

$$\Phi_0 = |\lfloor n/2 \rfloor - \lceil n/2 \rceil| \leq 1.$$

Considere la situación inmediatamente antes de una llamada a `balance()` que desplaza elementos y suponga, sin pérdida de generalidad, que `balance()`

está desplazando elementos porque $3 \cdot \text{front.size()} < \text{back.size()}.$ Observe que, en este caso,

$$\begin{aligned} n &= \text{front.size()} + \text{back.size()} \\ &< \text{back.size()}/3 + \text{back.size()} \\ &= \frac{4}{3}\text{back.size()} \end{aligned}$$

Además, el potencial en este punto en el tiempo es

$$\begin{aligned}
\Phi_1 &= \text{back.size()} - \text{front.size()} \\
&> \text{back.size()} - \text{back.size()}/3 \\
&= \frac{2}{3} \text{back.size()} \\
&> \frac{2}{3} \times \frac{3}{4} n \\
&= n/2
\end{aligned}$$

Por lo tanto, el número de llamadas a `add(i, x)` o `remove(i)` desde la última vez que `balance()` desplazó elementos es al menos $\Phi_1 - \Phi_0 > n/2 - 1$. Esto completa la prueba. $\square$

2.5.2 Resumen

El siguiente teorema resume las propiedades de un `DualArrayDeque`:

Teorema 2.4. *Un `DualArrayDeque` implementa la interfaz `List`. Ignorando el costo de las llamadas a `resize()` y `balance()`, un `DualArrayDeque` soporta las operaciones*

- `get(i)` y `set(i, x)` en tiempo $O(1)$ por operación; y
- `add(i, x)` y `remove(i)` en tiempo $O(1 + \min\{i, n - i\})$ por operación.

Además, comenzando con un `DualArrayDeque` vacío, cualquier secuencia de m operaciones `add(i, x)` y `remove(i)` resulta en un total de $O(m)$ tiempo empleado durante todas las llamadas a `resize()` y `balance()`.

2.6 RootishArrayStack: Una Pila Basada en Arreglos Eficiente en Espacio

Uno de los inconvenientes de todas las estructuras de datos anteriores en este capítulo es que, debido a que almacenan sus datos en uno o dos arreglos y evitan redimensionar estos arreglos con demasiada frecuencia, los arreglos con frecuencia no están muy llenos. Por ejemplo, inmediatamente después de una operación `resize()` en un `ArrayStack`, el arreglo de respaldo `a` solo está medio lleno. Peor aún, hay momentos en que solo un tercio de `a` contiene datos.

En esta sección, discutimos la estructura de datos `RootishArrayStack`, que aborda el problema del espacio desperdiciado. El `RootishArrayStack` almacena n elementos utilizando $O(\sqrt{n})$ arreglos. En estos arreglos, como máximo $O(\sqrt{n})$ ubicaciones de arreglo no se utilizan en ningún momento. Todas las ubicaciones de arreglo restantes se utilizan para almacenar datos. Por lo tanto, estas estructuras de datos desperdician como máximo $O(\sqrt{n})$ espacio al almacenar n elementos.

Un `RootishArrayStack` almacena sus elementos en una lista de r arreglos llamados *bloques* que están numerados $0, 1, \dots, r - 1$. Véase la Figura 2.5. El bloque `b` contiene `b+1` elementos. Por lo tanto, todos los r bloques contienen un total de

$$1 + 2 + 3 + \dots + r = r(r + 1)/2$$

elementos. La fórmula anterior se puede obtener como se muestra en la Figura 2.6.

RootishArrayStack

```
ArrayStack<T*> blocks;
int n;
```

Como era de esperar, los elementos de la lista se distribuyen en orden dentro de los bloques. El elemento de la lista con índice 0 se almacena en el bloque 0, los elementos con índices de lista 1 y 2 se almacenan en el bloque 1, los elementos con índices de lista 3, 4 y 5 se almacenan en el bloque 2, y así sucesivamente. El principal problema que tenemos que abordar es el de determinar, dado un índice i , qué bloque contiene i , así como el índice correspondiente a i dentro de ese bloque.

Determinar el índice de i dentro de su bloque resulta ser fácil. Si el índice i está en el bloque b , entonces el número de elementos en los bloques $0, \dots, b-1$ es $b(b+1)/2$. Por lo tanto, i se almacena en la ubicación

$$j = i - b(b+1)/2$$

img pag 56

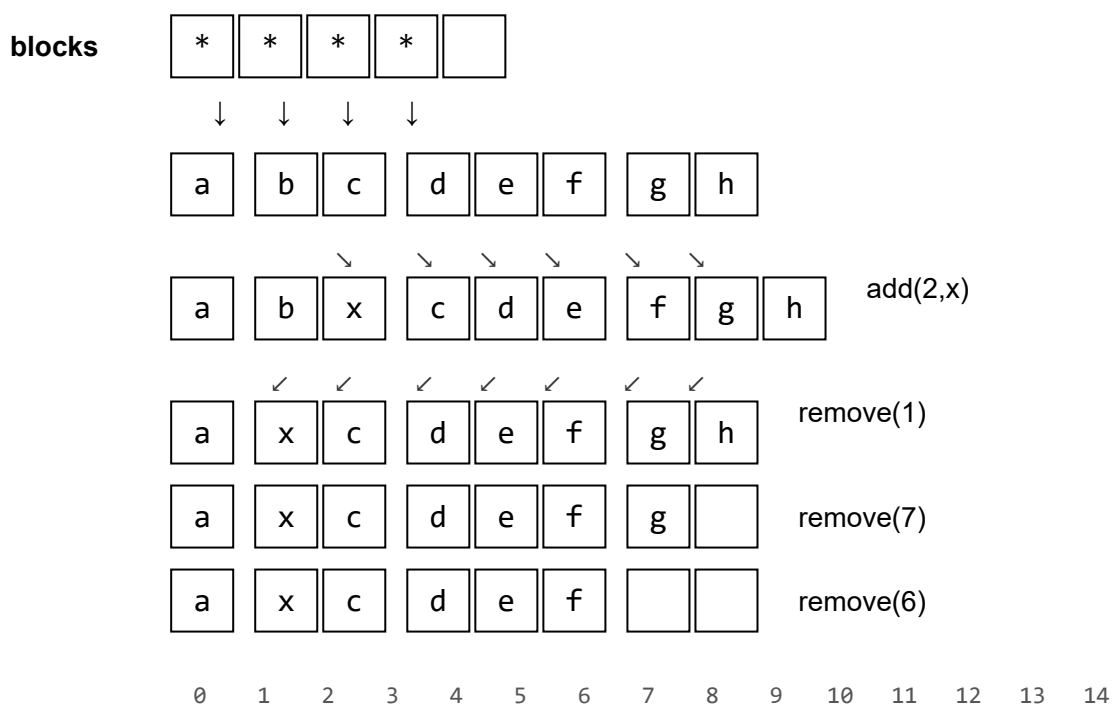


Figura 2.5: Una secuencia de operaciones `add(i,x)` y `remove(i)` en un `RootishArrayStack`. Las flechas denotan elementos que están siendo copiados.

img2 pag 56

Texto alternativo

Figura 2.6: El número de cuadrados blancos es $1 + 2 + 3 + \dots + r$. El número de cuadrados sombreados es el mismo. Juntos, los cuadrados blancos y sombreados forman un rectángulo que consta de $r(r+1)$ cuadrados.

dentro del bloque b . Un poco más desafiante es el problema de determinar el valor de b . El número de elementos que tienen índices menores o iguales a i es $i+1$. Por otro lado, el número de elementos en los bloques $0, \dots, b$ es $(b+1)(b+2)/2$. Por lo tanto, b es el entero más pequeño tal que

$$\frac{(b+1)(b+2)}{2} \geq i+1.$$

Podemos reescribir esta ecuación como

$$b^2 + 3b - 2i \geq 0.$$

La ecuación cuadrática correspondiente $b^2 + 3b - 2i = 0$ tiene dos soluciones: $b = (-3 + \sqrt{9 + 8i})/2$ y $b = (-3 - \sqrt{9 + 8i})/2$. La segunda solución no tiene sentido en nuestra aplicación, ya que siempre da un valor negativo. Por lo tanto, obtenemos la solución $b = (-3 + \sqrt{9 + 8i})/2$. En general, esta solución no es un entero, pero volviendo a nuestra desigualdad, queremos el entero más pequeño b tal que $b \geq (-3 + \sqrt{9 + 8i})/2$. Esto es simplemente

$$b = \left\lceil \frac{-3 + \sqrt{9 + 8i}}{2} \right\rceil.$$

RootishArrayStack

```
int i2b(int i) {
    double db = (-3.0 + sqrt(9 + 8*i)) / 2.0;
    int b = (int)ceil(db);
    return b;
}
```

Una vez resuelto esto, los métodos `get(i)` y `set(i,x)` son sencillos. Primero calculamos el bloque b apropiado y el índice j apropiado dentro del bloque y luego realizamos la operación apropiada:

RootishArrayStack

```
T get(int i) {
    int b = i2b(i);
    int j = i - b*(b+1)/2;
    return blocks.get(b)[j];
}

T set(int i, T x) {
    int b = i2b(i);
    int j = i - b*(b+1)/2;
    T y = blocks.get(b)[j];
    blocks.get(b)[j] = x;
    return y;
}
```

Si utilizamos cualquiera de las estructuras de datos de este capítulo para representar la lista `blocks`, entonces `get(i)` y `set(i,x)` se ejecutarán en tiempo constante.

El método `add(i,x)`, a estas alturas, nos resultará familiar. Primero verificamos si nuestra estructura de datos está llena, comprobando si el número de bloques, r , es tal que $r(r+1)/2 = n$. Si es así, llamamos a `grow()` para agregar otro bloque. Hecho esto, desplazamos los elementos con índices $i, \dots, n-1$ una posición a la derecha para hacer espacio para el nuevo elemento con índice i :

RootishArrayStack

```
void add(int i, T x) {
    int r = blocks.size();
    if ((r*(r+1)/2 < n + 1) grow();
    n++;
    for (int j = n-1; j > i; j--)
        set(j, get(j-1));
    set(i, x);
}
```

El método `grow()` hace lo que esperamos. Agrega un nuevo bloque:

RootishArrayStack

```
void grow() {
    blocks.add(blocks.size(), new T[blocks.size()+1]);
}
```

Ignorando el costo de la operación `grow()`, el costo de una operación `add(i,x)` está dominado por el costo del desplazamiento y, por lo tanto, es $O(1 + n - i)$, al igual que en un `ArrayStack`.

La operación `remove(i)` es similar a `add(i,x)`. Desplaza los elementos con índices `i+1, ..., n` una posición a la izquierda y luego, si hay más de un bloque vacío, llama al método `shrink()` para eliminar todos menos uno de los bloques no utilizados:

RootishArrayStack

```
T remove(int i) {
    T x = get(i);
    for (int j = i; j < n-1; j++)
        set(j, get(j+1));
    n--;
    int r = blocks.size();
    if ((r-2)*(r-1)/2 >= n) shrink();
    return x;
}
```

RootishArrayStack

```
void shrink() {
    int r = blocks.size();
    while (r > 0 && (r-2)*(r-1)/2 >= n) {
        delete [] blocks.remove(blocks.size()-1);
        r--;
    }
}
```

```
}
}
```

Una vez más, ignorando el costo de la operación `shrink()`, el costo de una operación `remove(i)` está dominado por el costo del desplazamiento y, por lo tanto, es $O(n - i)$.

2.6.1 Análisis del Crecimiento y la Contracción

El análisis anterior de `add(i, x)` y `remove(i)` no tiene en cuenta el costo de `grow()` y `shrink()`. Observe que, a diferencia de la operación `ArrayStack.resize()`, `grow()` y `shrink()` no copian ningún dato. Solo asignan o liberan un arreglo de tamaño r . En algunos entornos, esto toma solo tiempo constante, mientras que en otros, puede requerir tiempo proporcional a r .

Observamos que, inmediatamente después de una llamada a `grow()` o `shrink()`, la situación es clara. El bloque final está completamente vacío y todos los demás bloques están completamente llenos. Otra llamada a `grow()` o `shrink()` no ocurrirá hasta que se hayan agregado o eliminado al menos $r - 1$ elementos. Por lo tanto, incluso si `grow()` y `shrink()` toman tiempo $O(r)$, este costo se puede amortizar sobre al menos $r - 1$ operaciones `add(i, x)` o `remove(i)`, de modo que el costo amortizado de `grow()` y `shrink()` es $O(1)$ por operación.

2.6.2 Uso del Espacio

A continuación, analizamos la cantidad de espacio extra utilizado por un `RootishArrayStack`. En particular, queremos contar cualquier espacio utilizado por un `RootishArrayStack` que no sea un elemento de arreglo utilizado actualmente para contener un elemento de la lista. A todo ese espacio lo llamamos espacio desperdiciado.

La operación `remove(i)` asegura que un `RootishArrayStack` nunca tenga más de dos bloques que no estén completamente llenos. El número de bloques, r , utilizado por un `RootishArrayStack` que almacena n elementos, por lo tanto, satisface

$$(r - 2)(r - 1) \leq n.$$

Nuevamente, usando la ecuación cuadrática en esto se obtiene

$$r \leq \frac{3 + \sqrt{1 + 4n}}{2} = O(\sqrt{n}).$$

Los dos últimos bloques tienen tamaños r y $r - 1$, por lo que el espacio desperdiciado por estos dos bloques es como máximo $2r - 1 = O(\sqrt{n})$. Si almacenamos los bloques en (por ejemplo) un `ArrayStack`, entonces la cantidad de espacio desperdiciado por la `List` que almacena esos r bloques también es $O(r) = O(\sqrt{n})$. El otro espacio necesario para almacenar n y otra información contable es $O(1)$. Por lo tanto, la cantidad total de espacio desperdiciado en un `RootishArrayStack` es $O(\sqrt{n})$.

A continuación, argumentamos que este uso del espacio es óptimo para cualquier estructura de datos que comience vacía y pueda soportar la adición de un elemento a la vez. Más precisamente, mostraremos que, en algún momento durante la adición de n elementos, la estructura de datos está desperdiciando una cantidad de espacio de al menos $\sqrt{n}$ (aunque puede ser desperdiciado solo por un momento).

Supongamos que comenzamos con una estructura de datos vacía y agregamos n elementos uno a la vez. Al final de este proceso, los n elementos se almacenan en la estructura y se distribuyen entre una colección de r bloques de memoria. Si $r \geq \sqrt{n}$, entonces la estructura de datos debe estar usando r punteros (o referencias) para realizar un seguimiento de estos r bloques, y estos punteros son espacio desperdiciado. Por otro lado, si $r < \sqrt{n}$, entonces por el principio del palomar, algún bloque debe tener un tamaño de al menos $n/r > \sqrt{n}$. Consideremos el momento en que este bloque fue asignado por primera vez. Inmediatamente después de ser asignado, este bloque estaba vacío y, por lo tanto, desperdiciaba $\sqrt{n}$ espacio. Por lo tanto, en algún momento durante la inserción de n elementos, la estructura de datos estaba desperdiciando $\sqrt{n}$ espacio.

2.6.3 Resumen

El siguiente teorema resume nuestra discusión sobre la estructura de datos `RootishArrayStack`:

Teorema 2.5. *Un `RootishArrayStack` implementa la interfaz `List`. Ignorando el costo de las llamadas a `grow()` y `shrink()`, un `RootishArrayStack` soporta las operaciones*

- `get(i)` y `set(i, x)` en tiempo $O(1)$ por operación; y
- `add(i, x)` y `remove(i)` en tiempo $O(1 + n - i)$ por operación.

Además, comenzando con un `RootishArrayStack` vacío, cualquier secuencia de m operaciones `add(i, x)` y `remove(i)` resulta en un total de $O(m)$ tiempo empleado durante todas las llamadas a `grow()` y `shrink()`.

El espacio (medido en palabras)³ utilizado por un `RootishArrayStack` que almacena n elementos es $n + O(\sqrt{n})$.

2.6.4 Cálculo de Raíces Cuadradas

Un lector que haya tenido cierta exposición a los modelos de computación puede notar que el `RootishArrayStack`, como se describió anteriormente, no encaja en el modelo habitual de *word-RAM* (Sección 1.4) porque requiere calcular raíces cuadradas. La operación de raíz cuadrada generalmente no se considera una operación básica y, por lo tanto, no suele ser parte del modelo *word-RAM*.

En esta sección, mostramos que la operación de raíz cuadrada se puede implementar de manera eficiente. En particular, mostramos que para cualquier entero $x \in \{0, \dots, n\}$, $\lfloor \sqrt{x} \rfloor$ se puede calcular en tiempo constante, después de un preprocesamiento $O(\sqrt{n})$ que crea dos arreglos de longitud $O(\sqrt{n})$. El siguiente lema muestra que podemos reducir el problema de calcular la raíz cuadrada de x a la raíz cuadrada de un valor relacionado x' .

Lema 2.3. *Sea $x \geq 1$ y sea $x' = x - a$, donde $0 \leq a \leq \sqrt{x}$. Entonces $\sqrt{x'} \geq \sqrt{x} - 1$.*

³Recuerde la Sección 1.4 para una discusión sobre cómo se mide la memoria.

Prueba. Es suficiente demostrar que

$$\sqrt{x} - \sqrt{x'} \leq 1.$$

Eleva al cuadrado ambos lados de esta desigualdad para obtener

$$x - \sqrt{x} \geq x - 2\sqrt{x} + 1$$

y agrupa los términos para obtener

$$\sqrt{x} \geq 1$$

lo cual es claramente cierto para cualquier $x \geq 1$. $\square$

Comienza restringiendo un poco el problema, y suponga que $2^r \leq x < 2^{r+1}$, de modo que $\lfloor \log x \rfloor = r$, es decir, x es un entero que tiene $r + 1$ bits en su representación binaria. Podemos tomar $x' = x - (x \bmod 2^{\lfloor r/2 \rfloor})$. Ahora, x' satisface las condiciones del Lema 2.3, por lo que $\sqrt{x} - \sqrt{x'} \leq 1$. Además, x' tiene todos sus bits de orden inferior $\lfloor r/2 \rfloor$ iguales a 0, por lo que solo hay

$$2^{r+1-\lfloor r/2 \rfloor} \leq 4 \cdot 2^{r/2} \leq 4\sqrt{x}$$

valores posibles de x' . Esto significa que podemos usar un arreglo, `sqrttab`, que almacene el valor de $\lfloor \sqrt{x'} \rfloor$ para cada valor posible de x' . Un poco más precisamente, tenemos

$$\text{sqrttab}[i] = \lfloor \sqrt{i2^{\lfloor r/2 \rfloor}} \rfloor.$$

De esta manera, `sqrttab[i]` está dentro de 2 de $\sqrt{x'}$ para todo $x' \in \{i2^{\lfloor r/2 \rfloor}, \dots, (i+1)2^{\lfloor r/2 \rfloor} - 1\}$. Dicho de otra manera, la entrada del arreglo $s = \text{sqrttab}[x \gg \lfloor r/2 \rfloor]$ es igual a $\lfloor \sqrt{x'} \rfloor$, $\lfloor \sqrt{x'} \rfloor - 1$, o $\lfloor \sqrt{x'} \rfloor - 2$. A partir de s podemos determinar el valor de $\lfloor \sqrt{x} \rfloor$ incrementando s hasta que $(s+1)^2 > x$.

FastSqrt

```
int sqrt(int x, int r) {
    int s = sqrttab[x>>r/2];
    while ((s+1)*(s+1) <= x) s++; // se ejecuta como máximo dos veces
    return s;
}
```

Ahora, esto solo funciona para $x \in [2^r, \dots, 2^{r+1} - 1]$ y `sqrttab` es una tabla especial que solo funciona para un valor particular de $r = \lfloor \log x \rfloor$. Para superar esto, podríamos calcular $\lfloor \log n \rfloor$ diferentes arreglos `sqrttab`, uno para cada posible valor de $\lfloor \log x \rfloor$. Los tamaños de estas tablas forman una secuencia exponencial cuyo valor más grande es como máximo $4\sqrt{n}$, por lo que el tamaño total de todas las tablas es $O(\sqrt{n})$.

Sin embargo, resulta que más de un arreglo `sqrttab` es innecesario; solo necesitamos un arreglo `sqrttab` para el valor $r = \lfloor \log n \rfloor$. Cualquier valor x con $\log x = r' < r$ puede ser actualizado multiplicando x por $2^{r-r'}$ y usando la ecuación

$$\sqrt{2^{r-r'}x} = 2^{(r-r')/2}\sqrt{x}.$$

La cantidad $2^{r-r'}x$ está en el rango $[2^r, \dots, 2^{r+1} - 1]$ por lo que podemos buscar su raíz cuadrada en `sqrttab`. El siguiente código implementa esta idea para calcular $\lfloor \sqrt{x} \rfloor$ para todos los enteros no negativos x en el rango $[0, \dots, 2^{30} - 1]$ utilizando un arreglo, `sqrttab`, de tamaño 2^{16} .

FastSqrt

```
int sqrt(int x) {
    int rp = log(x);
    int upgrade = ((r-rp)/2) * 2;
    int xp = x << upgrade; // xp tiene r o r-1 bits
    int s = sqrttab[xp>>(r/2)] >> (upgrade/2);
    while ((s+1)*(s+1) <= x) s++; // se ejecuta como máximo dos veces
    return s;
}
```

Algo que hemos dado por sentado hasta ahora es la cuestión de cómo calcular $r' = \lfloor \log x \rfloor$. De nuevo, este es un problema que se puede resolver con un arreglo, `logtab`, de tamaño $2^{r/2}$. En este caso, el código es particularmente simple, ya que $\lfloor \log x \rfloor$ es solo el índice del bit 1 más significativo en la representación binaria de x . Esto significa que, para $x > 2^{r/2}$, podemos desplazar a la derecha los bits de x en $r/2$ posiciones antes de usarlo como un índice en `logtab`. El siguiente código hace esto utilizando un arreglo `logtab` de tamaño 2^{16} para calcular $\lfloor \log x \rfloor$ para todos los x en el rango $[1, \dots, 2^{32} - 1]$.

FastSqrt

```
int log(int x) {
    if (x >= halfint)
        return 16 + logtab[x>>16];
    return logtab[x];
}
```

Finalmente, para completar, incluimos el siguiente código que inicializa `logtab` y `sqrttab`:

FastSqrt

```
void inittabs() {
    sqrttab = new int[1<<(r/2)];
    logtab = new int[1<<(r/2)];
    for (int d = 0; d < r/2; d++)
        for (int k = 0; k < 1<<d; k++)
            logtab[1<<d+k] = d;
    int s = 1<<(r/4); // raíz cuadrada(2^(r/2))
    for (int i = 0; i < 1<<(r/2); i++) {
        if ((s+1)*(s+1) <= i < (r/2)) s++; // la raíz cuadrada aumenta
        sqrttab[i] = s;
    }
}
```

En resumen, los cálculos realizados por el método `i2b(i)` se pueden implementar en tiempo constante en la *word-RAM* utilizando $O(\sqrt{n})$ memoria adicional para almacenar los arreglos `sqrttab` y `logtab`. Estos arreglos se pueden reconstruir cuando n aumenta o disminuye por un factor de dos, y el costo de esta reconstrucción se puede amortizar sobre el número de operaciones `add(i, x)` y `remove(i)` que causaron el cambio en n , de la misma manera que se analiza el costo de `resize()` en la implementación de `ArrayStack`.

2.7 Discusión y Ejercicios

La mayoría de las estructuras de datos descritas en este capítulo son folclore. Se pueden encontrar en implementaciones que datan de hace más de 30 años. Por ejemplo, las implementaciones de pilas, colas y deque, que se generalizan fácilmente a las estructuras `ArrayStack`, `ArrayQueue` y `ArrayDeque` descritas aquí, son discutidas por Knuth [46, Sección 2.2.2].

Brodnik et al. [13] parecen haber sido los primeros en describir el `RootishArrayStack` y probar una cota inferior de $\sqrt{n}$ como la de la Sección 2.6.2. También presentan una estructura diferente que utiliza una elección más sofisticada de tamaños de bloque para evitar calcular raíces cuadradas en el método `i2b(i)`. Dentro de su esquema, el bloque que contiene `i` es el bloque `floor(log(i + 1))`, que es simplemente el índice del bit 1 principal en la representación binaria de `i + 1`. Algunas arquitecturas de computadoras proporcionan una instrucción para calcular el índice del bit 1 principal en un entero.

Una estructura relacionada con el `RootishArrayStack` es el vector escalonado de dos niveles (*two-level tiered-vector*) de Goodrich y Kloss [35]. Esta estructura soporta las operaciones `get(i, x)` y `set(i, x)` en tiempo constante y `add(i, x)` y `remove(i)` en tiempo $O(\sqrt{n})$. Estos tiempos de ejecución son similares a lo que se puede lograr con la implementación más cuidadosa de un `RootishArrayStack` discutida en el Ejercicio 2.10.

Ejercicio 2.1. El método `List addAll(i, c)` inserta todos los elementos de la `Collection c` en la lista en la posición `i`. (El método `add(i, x)` es un caso especial donde `c = {x}`.) Explique por qué, para las estructuras de datos de este capítulo, no es eficiente implementar `addAll(i, c)` mediante llamadas repetidas a `add(i, x)`. Diseñe e implemente una implementación más eficiente.

Ejercicio 2.2. Diseñe e implemente una `RandomQueue`. Esta es una implementación de la interfaz `Queue` en la que la operación `remove()` elimina un elemento elegido uniformemente al azar entre todos los elementos actualmente en la cola. (Piense en una `RandomQueue` como una bolsa en la que podemos agregar elementos o alcanzar y eliminar ciegamente algún elemento aleatorio.) Las operaciones `add(x)` y `remove()` en una `RandomQueue` deben ejecutarse en tiempo constante por operación.

Ejercicio 2.3. Diseñe e implemente una `Treque` (cola de triple extremo). Esta es una implementación de `List` en la que `get(i)` y `set(i, x)` se ejecutan en tiempo constante y `add(i, x)` y `remove(i)` se ejecutan en tiempo

$$O(1 + \min\{i, n - i, |n/2 - i|\}).$$

En otras palabras, las modificaciones son rápidas si están cerca de cualquiera de los extremos o cerca de la mitad de la lista.

Ejercicio 2.4. Implemente un método `rotate(a, r)` que "rote" el arreglo `a` de modo que `a[i]` se mueva a `a[(i + r) mod a.length]`, para todo $i \in \{0, \dots, a.length\}$.

Ejercicio 2.5. Implemente un método `rotate(r)` que "rote" una `List` de modo que el elemento de la lista `i` se convierta en el elemento de la lista $(i + r) \bmod n$. Cuando se ejecuta en un `ArrayDeque`, o un `DualArrayDeque`, `rotate(r)` debe ejecutarse en tiempo $O(1 + \min\{r, n - r\})$.

Ejercicio 2.6. Modifique la implementación de `ArrayDeque` para que el desplazamiento realizado por `add(i, x)`, `remove(i)` y `resize()` se realice utilizando el método más rápido `System.arraycopy(s, i, d, j, n)`.

Ejercicio 2.7. Modifique la implementación de `ArrayDeque` para que no utilice el operador `%` (que es costoso en algunos sistemas). En su lugar, debería hacer uso del hecho de que, si `a.length` es una potencia de 2, entonces

$$k \% a.length = k \& (a.length - 1).$$

(Aquí, `&` es el operador bit a bit AND.)

Ejercicio 2.8. Diseñe e implemente una variante de `ArrayDeque` que no realice ninguna aritmética modular. En su lugar, todos los datos se ubican en un bloque contiguo, en orden, dentro de un arreglo. Cuando los datos sobrepasan el principio o el final de este arreglo, se realiza una operación `rebuild()` modificada. El costo amortizado de todas las operaciones debería ser el mismo que en un `ArrayDeque`.

Sugerencia: Lograr que esto funcione se trata realmente de cómo se implementa la operación `rebuild()`. Uno querría que `rebuild()` colocara la estructura de datos en un estado en el que los datos no puedan salirse por ningún extremo hasta que se hayan realizado al menos $n/2$ operaciones.

Pruebe el rendimiento de su implementación contra el `ArrayDeque`. Optimice su implementación (usando `System.arraycopy(a, i, b, i, n)`) y vea si puede superar la implementación de `ArrayDeque`.

Ejercicio 2.9. Diseñe e implemente una versión de un `RootishArrayStack` que solo tenga $O(\sqrt{n})$ espacio desperdiciado, pero que pueda realizar operaciones `add(i, x)` y `remove(i, x)` en tiempo $O(1 + \min\{i, n - i\})$.

Ejercicio 2.10. Diseñe e implemente una versión de un `RootishArrayStack` que solo tenga $O(\sqrt{n})$ espacio desperdiciado, pero que pueda realizar operaciones `add(i, x)` y `remove(i, x)` en tiempo $O(1 + \min\{\sqrt{n}, n - i\})$. (Para una idea de cómo hacer esto, véase la Sección 3.3.)

Ejercicio 2.11. Diseñe e implemente una versión de un `RootishArrayStack` que solo tenga $O(\sqrt{n})$ espacio desperdiciado, pero que pueda realizar operaciones `add(i, x)` y `remove(i, x)` en tiempo $O(1 + \min\{i, \sqrt{n}, n - i\})$. (Véase la Sección 3.3 para ideas sobre cómo lograr esto.)

Ejercicio 2.12. Diseñe e implemente un `CubishArrayStack`. Esta estructura de tres niveles implementa la interfaz `List` utilizando $O(n^{2/3})$ espacio desperdiciado. En esta estructura, `get(i)` y `set(i, x)` toman tiempo constante; mientras que `add(i, x)` y `remove(i)` toman tiempo amortizado $O(n^{1/3})$.

Capítulo 3

Listas Enlazadas

En este capítulo, continuamos estudiando implementaciones de la interfaz `List`, esta vez utilizando estructuras de datos basadas en punteros en lugar de arreglos. Las estructuras de este capítulo están formadas por nodos que contienen los elementos de la lista. Utilizando referencias (punteros), los nodos se enlazan entre sí formando una secuencia. Primero estudiamos las listas simplemente enlazadas, que pueden implementar operaciones de Pila (*Stack*) y Cola (FIFO) en tiempo constante por operación, y luego pasamos a las listas doblemente enlazadas, que pueden implementar operaciones de Deque en tiempo constante.

Las listas enlazadas tienen ventajas y desventajas en comparación con las implementaciones basadas en arreglos de la interfaz `List`. La principal desventaja es que perdemos la capacidad de acceder a cualquier

elemento utilizando `get(i)` o `set(i,x)` en tiempo constante. En su lugar, tenemos que recorrer la lista, un elemento a la vez, hasta llegar al i -ésimo elemento. La principal ventaja es que son más dinámicas: Dada una referencia a cualquier nodo `u` de la lista, podemos eliminar `u` o insertar un nodo adyacente a `u` en tiempo constante. Esto es cierto sin importar dónde se encuentre `u` en la lista.

3.1 SLList: Una Lista Simplemente Enlazada

Una `SLList` (lista simplemente enlazada) es una secuencia de `Nodes` (nodos). Cada nodo `u` almacena un valor de dato `u.x` y una referencia `u.next` al siguiente nodo en la secuencia. Para el último nodo `w` en la secuencia, `w.next = null`



Figura 3.1: Una secuencia de operaciones de Cola (`add(x)` y `remove()`) y Pila (`push(x)` y `pop()`) en una `SLList`.

SLList

```
class Node {
public:
    T x;
    Node *next;
    Node(T x0) {
        x = x0;
        next = NULL;
    }
};
```

Por eficiencia, una `SLList` utiliza las variables `head` y `tail` para llevar un registro del primer y último nodo en la secuencia, así como un entero `n` para llevar un registro de la longitud de la secuencia:

SLList

```
Node *head;
Node *tail;
int n;
```

Una secuencia de operaciones de Pila y Cola en una `SLList` se ilustra en la Figura 3.1.

Una `SLList` puede implementar eficientemente las operaciones de Pila `push()` y `pop()` agregando y eliminando elementos en la cabeza de la secuencia. La operación `push()` simplemente crea un nuevo nodo `u` con valor de dato `x`, establece `u.next` en la antigua cabeza de la lista y hace que `u` sea la nueva cabeza de la lista. Finalmente, incrementa `n` ya que el tamaño de la `SLList` ha aumentado en uno:

SLList

```

T push(T x) {
    Node *u = new Node(x);
    u->next = head;
    head = u;
    if (n == 0)
        tail = u;
    n++;
    return x;
}

```

La operación `pop()`, después de verificar que la `SLList` no está vacía, elimina la cabeza estableciendo `head = head.next` y decrementando `n`. Un caso especial ocurre cuando se está eliminando el último elemento, en cuyo caso `tail` se establece en `null`:

```

T pop() {
    if (n == 0) return null;
    T x = head->x;
    Node *u = head;
    head = head->next;
    delete u;
    if (--n == 0) tail = NULL;
    return x;
}

```

Claramente, tanto las operaciones `push(x)` como `pop()` se ejecutan en tiempo $O(1)$.

3.1.1 Operaciones de Cola

Una `SLList` también puede implementar las operaciones de cola FIFO `add(x)` y `remove()` en tiempo constante. Las eliminaciones se realizan desde la cabeza de la lista, y son idénticas a la operación `pop()`:

SLList

```

T remove() {
    if (n == 0) return null;
    T x = head->x;
    Node *u = head;
    head = head->next;
    delete u;
    if (--n == 0) tail = NULL;
    return x;
}

```

Las adiciones, por otro lado, se realizan al final de la lista. En la mayoría de los casos, esto se hace estableciendo `tail.next = u`, donde `u` es el nodo recién creado que contiene `x`. Sin embargo, ocurre un

caso especial cuando $n = 0$, en cuyo caso $\text{tail} = \text{head} = \text{null}$. En este caso, tanto tail como head se establecen en u .

SLList

```
bool add(T x) {
    Node *u = new Node(x);
    if (n == 0) {
        head = u;
    } else {
        tail->next = u;
    }
    tail = u;
    n++;
    return true;
}
```

Claramente, tanto $\text{add}(x)$ como $\text{remove}()$ toman tiempo constante.

3.1.2 Resumen

El siguiente teorema resume el rendimiento de una **SLList**:

Teorema 3.1. Una **SLList** implementa las interfaces de Pila (*Stack*) y Cola (*FIFO*). Las operaciones $\text{push}(x)$, $\text{pop}()$, $\text{add}(x)$ y $\text{remove}()$ se ejecutan en tiempo $O(1)$ por operación.

Una **SLList** casi implementa el conjunto completo de operaciones de Deque. La única operación que falta es la eliminación desde la cola de una **SLList**. Eliminar desde la cola de una **SLList** es difícil porque requiere actualizar el valor de tail para que apunte al nodo w que precede a tail en la **SLList**; este es el nodo w tal que $w.\text{next} = \text{tail}$. Desafortunadamente, la única forma de llegar a w es recorriendo la **SLList** comenzando en head y dando $n - 2$ pasos.

3.2 DLList: Una Lista Doblemente Enlazada

Una **DLList** (lista doblemente enlazada) es muy similar a una **SLList**, excepto que cada nodo u en una **DLList** tiene referencias tanto al nodo $u.\text{next}$ que le sigue como al nodo $u.\text{prev}$ que le precede.

DLList

```
struct Node {
    T x;
    Node *prev, *next;
};
```

Al implementar una **SLList**, vimos que siempre había varios casos especiales de los que preocuparse. Por ejemplo, eliminar el último elemento de una **SLList** o agregar un elemento a una **SLList** vacía requiere tener cuidado para asegurar que head y tail se actualicen correctamente. En una **DLList**, el número de estos casos especiales aumenta considerablemente. Quizás la forma más limpia de manejar todos estos casos

especiales en una **DLList** es introducir un nodo **dummy** (ficticio). Este es un nodo que no contiene ningún dato, sino que actúa como un marcador de posición para que no haya nodos especiales; cada nodo tiene tanto un **next** como un **prev**, con **dummy** actuando como el nodo que sigue al último nodo de la lista y que precede al primer nodo de la lista. De esta manera, los nodos de la lista están (doblemente) enlazados formando un ciclo, como se ilustra en la Figura 3.2.

DLList

```
Node dummy;
int n;
DLList() {
    dummy.next = &dummy;
    dummy.prev = &dummy;
    n = 0;
}
```



```
}
```

Encontrar el nodo con un índice particular en una **DLList** es fácil; podemos comenzar en la cabeza de la lista (**dummy.next**) y avanzar, o comenzar en la cola de la lista (**dummy.prev**) y retroceder. Esto nos permite alcanzar el i -ésimo nodo en tiempo $O(1 + \min\{i, n - i\})$:

DLList

```
Node* getNode(int i) {
    Node* p;
    if (i < n / 2) {
        p = dummy.next;
        for (int j = 0; j < i; j++)
            p = p->next;
    } else {
        p = &dummy;
        for (int j = n; j > i; j--)
            p = p->prev;
    }
    return (p);
}
```

Las operaciones **get(i)** y **set(i,x)** ahora también son sencillas. Primero encontramos el i -ésimo nodo y luego obtenemos o establecemos su valor **x**:

DLList

```
T get(int i) {
    return getNode(i)->x;
}
T set(int i, T x) {
    Node* u = getNode(i);
```



```
    T y = u->x;
    u->x = x;
    return y;
}
```

El tiempo de ejecución de estas operaciones está dominado por el tiempo que toma encontrar el i -ésimo nodo, y por lo tanto es $O(1 + \min\{i, n - i\})$.

3.2.1 Agregar y Eliminar

Si tenemos una referencia a un nodo w en una `DLList` y queremos insertar un nodo u antes de w , entonces esto es solo cuestión de establecer $u.next = w$, $u.prev = w.prev$, y luego ajustar $u.prev.next$ y $u.next.prev$. (Véase la Figura 3.3.) Gracias al nodo ficticio (*dummy*), no hay necesidad de preocuparse por si $w.prev$ o $w.next$ no existen.

DLList

```
Node* addBefore(Node *w, T x) {
    Node *u = new Node;
    u->x = x;
    u->prev = w->prev;
    u->next = w;
    u->next->prev = u;
    u->prev->next = u;
    n++;
    return u;
}
```

Ahora, la operación de lista `add(i, x)` es trivial de implementar. Encontramos el i -ésimo nodo en la `DLList` e insertamos un nuevo nodo u que contiene x justo antes de él.

DLList

```
void add(int i, T x) {
    addBefore(getNode(i), x);
}
```

La única parte no constante del tiempo de ejecución de `add(i, x)` es el tiempo que toma encontrar el i -ésimo nodo (usando `getNode(i)`). Por lo tanto, `add(i, x)` se ejecuta en tiempo $O(1 + \min\{i, n - i\})$.

Eliminar un nodo `w` de una `DLList` es fácil. Solo necesitamos ajustar los punteros en `w.next` y `w.prev` para que se salten `w`. De nuevo, el uso del nodo ficticio (`dummy`) elimina la necesidad de considerar cualquier caso especial:

DLList

```
void remove(Node *w) {
    w->prev->next = w->next;
    w->next->prev = w->prev;
    delete w;
    n--;
}
```

Ahora la operación `remove(i)` es trivial. Encontramos el nodo con índice i y lo eliminamos:

DLList

```
T remove(int i) {
    Node *w = getNode(i);
    T x = w->x;
    remove(w);
    return x;
}
```

Nuevamente, la única parte costosa de esta operación es encontrar el i -ésimo nodo usando `getNode(i)`, por lo que `remove(i)` se ejecuta en tiempo $O(1 + \min\{i, n - i\})$.

3.2.2 Resumen

El siguiente teorema resume el rendimiento de una `DLList`:

Teorema 3.2. Una `DLList` implementa la interfaz `List`. En esta implementación, las operaciones `get(i)`, `set(i, x)`, `add(i, x)` y `remove(i)` se ejecutan en tiempo $O(1 + \min\{i, n - i\})$ por operación.

Vale la pena señalar que, si ignoramos el costo de la operación `getNode(i)`, entonces todas las operaciones en una `DLList` toman tiempo constante. Por lo tanto, la única parte costosa de las operaciones en una `DLList` es encontrar el nodo relevante. Una vez que tenemos el nodo relevante, agregar, eliminar o acceder a los datos en ese nodo toma solo tiempo constante.

Esto está en marcado contraste con las implementaciones de `List` basadas en arreglos del Capítulo 2; en esas implementaciones, el elemento relevante del arreglo se puede encontrar en tiempo constante. Sin embargo, la adición o eliminación requiere desplazar elementos en el arreglo y, en general, toma un tiempo no constante.

Por esta razón, las estructuras de listas enlazadas son adecuadas para aplicaciones donde se pueden obtener referencias a los nodos de la lista a través de medios externos. Por ejemplo, los punteros a los nodos de una

lista enlazada podrían almacenarse en un **USet**. Entonces, para eliminar un elemento **x** de la lista enlazada, el nodo que contiene **x** se puede encontrar rápidamente usando el **USet** y el nodo se puede eliminar de la lista en tiempo constante.

3.3 SList: Una Lista Enlazada Eficiente en Espacio

Uno de los inconvenientes de las listas enlazadas (además del tiempo que toma acceder a elementos que están en lo profundo de la lista) es su uso del espacio. Cada nodo en una **DLList** requiere dos referencias adicionales a los nodos siguiente y anterior en la lista. ¡Dos de los campos en un **Node** están dedicados a mantener la lista, y solo uno de los campos es para almacenar datos!

Una **SList** (lista eficiente en espacio) reduce este espacio desperdiciado utilizando una idea simple: En lugar de almacenar elementos individuales en una **DLList**, almacenamos un bloque (arreglo) que contiene varios elementos. Más precisamente, una **SList** está parametrizada por un tamaño de bloque **b**. Cada nodo individual en una **SList** almacena un bloque que puede contener hasta **b + 1** elementos.

Por razones que se aclararán más adelante, será útil si podemos realizar operaciones de Deque en cada bloque. La estructura de datos que elegimos para esto es un **BDeque** (deque acotado), derivado de la estructura **ArrayDeque** descrita en la Sección 2.4. El **BDeque** difiere del **ArrayDeque** en un pequeño aspecto: Cuando se crea un nuevo **BDeque**, el tamaño del arreglo de respaldo **a** se fija en **b + 1** y nunca crece ni se contrae. La propiedad importante de un **BDeque** es que permite la adición o eliminación de elementos tanto al frente como al final en tiempo constante. Esto será útil a medida que los elementos se desplazan de un bloque a otro.

SList

```
class BDeque : public ArrayDeque<T> {
public:
    BDeque(int b) {
        n = 0;
        j = 0;
        array<int> z(b+1);
        a = z;
    }
    ~BDeque() { }
    // Pregunta de C++: ¿Por qué es esto necesario?
    void add(int i, T x) {
        ArrayDeque<T>::add(i, x);
    }
    bool add(T x) {
        ArrayDeque<T>::add(size(), x);
        return true;
    }
    void resize() { }
};
```

Una SList es entonces una lista doblemente enlazada de bloques:

SList

```
class Node {
public:
    BDeque d;
    Node *prev, *next;
    Node(int b) : d(b) { }
};
```

SEList

```
int n;
Node dummy;
```

3.3.1 Requisitos de Espacio

Una **SEList** impone restricciones muy estrictas sobre el número de elementos en un bloque: A menos que un bloque sea el último bloque, entonces ese bloque contiene al menos $b - 1$ y como máximo $b + 1$ elementos. Esto significa que, si una **SEList** contiene n elementos, entonces tiene como máximo

$$n/(b - 1) + 1 = O(n/b)$$

bloques. El **BDeque** para cada bloque contiene un arreglo de longitud $b + 1$, pero para cada bloque excepto el último, se desperdicia a lo sumo una cantidad constante de espacio en este arreglo. La memoria restante utilizada por un bloque también es constante. Esto significa que el espacio desperdiciado en una **SEList** es solo $O(b + n/b)$. Al elegir un valor de b dentro de un factor constante de $\sqrt{n}$, podemos hacer que el exceso de espacio de una **SEList** se aproxime a la cota inferior de $\sqrt{n}$ dada en la Sección 2.6.2.

3.3.2 Encontrar Elementos

El primer desafío que enfrentamos con una **SEList** es encontrar el elemento de la lista con un índice i dado. Tenga en cuenta que la ubicación de un elemento consta de dos partes:

1. El nodo u que contiene el bloque que contiene el elemento con índice i ; y
2. El índice j del elemento dentro de su bloque.

SEList

```
class Location {
public:
    Node *u;
    int j;
    Location() { }
    Location(Node *u, int j) {
        this->u = u;
        this->j = j;
    }
};
```

Para encontrar el bloque que contiene un elemento en particular, procedemos de la misma manera que lo hacemos en una `DLList`. O comenzamos al frente de la lista y recorremos en dirección hacia adelante, o en la parte posterior de la lista y recorremos hacia atrás hasta que llegamos al nodo que queremos. La única diferencia es que, cada vez que pasamos de un nodo al siguiente, saltamos un bloque completo de elementos.

SEList

```
void getLocation(int i, Location &ell) {
    if (i < n / 2) {
        Node *u = dummy.next;
        while (i >= u->d.size()) {
            i -= u->d.size();
            u = u->next;
        }
        ell.u = u;
        ell.j = i;
    } else {
        Node *u = &dummy;
        int idx = n;
        while (i < idx) {
            u = u->prev;
            idx -= u->d.size();
        }
        ell.u = u;
        ell.j = i - idx;
    }
}
```

Recuerde que, con la excepción de como máximo un bloque, cada bloque contiene al menos $b - 1$ elementos, por lo que cada paso en nuestra búsqueda nos acerca $b - 1$ elementos al elemento que estamos buscando. Si buscamos hacia adelante, esto significa que llegamos al nodo que queremos después de $O(1 + i/b)$ pasos. Si buscamos hacia atrás, entonces llegamos al nodo que queremos después de $O(1 + (n - i)/b)$ pasos. El algoritmo toma la menor de estas dos cantidades dependiendo del valor de i , por lo que el tiempo para localizar el elemento con índice i es $O(1 + \min\{i, n - i\}/b)$.

Una vez que sabemos cómo localizar el elemento con índice i , las operaciones `get(i)` y `set(i, x)` se traducen en obtener o establecer un índice particular en el bloque correcto:

SEList

```
T get(int i) {
    Location l;
    getLocation(i, l);
    return l.u->d.get(l.j);
}

T set(int i, T x) {
    Location l;
    getLocation(i, l);
```

```

    T y = l.u->d.get(l.j);
    l.u->d.set(l.j, x);
    return y;
}

```

Los tiempos de ejecución de estas operaciones están dominados por el tiempo que toma localizar el elemento, por lo que también se ejecutan en tiempo $O(1 + \min\{i, n - i\}/b)$.

3.3.3 Agregar un Elemento

Agregar elementos a una `SEList` es un poco más complicado. Antes de considerar el caso general, consideramos la operación más fácil, `add(x)`, en la cual `x` se agrega al final de la lista. Si el último bloque está lleno (o no existe porque todavía no hay bloques), entonces primero asignamos un nuevo bloque y lo añadimos a la lista de bloques. Ahora que estamos seguros de que el último bloque existe y no está lleno, agregamos `x` al último bloque.

```

void add(T x) {
    Node *last = dummy.prev;
    if (last == &dummy || last->d.size() == b+1) {
        last = addBefore(&dummy);
    }
    last->d.add(x);
    n++;
}

```

Las cosas se complican más cuando agregamos al interior de la lista usando `add(i, x)`. Primero localizamos `i` para obtener el nodo `u` cuyo bloque contiene



el i -ésimo elemento de la lista. El problema es que queremos insertar `x` en el bloque de `u`, pero tenemos que estar preparados para el caso en que el bloque de `u` ya contenga `b+1` elementos, de modo que esté lleno y no haya espacio para `x`.

Sean $u_0, u_1, u_2, \dots$ denotan `u`, `u.next`, `u.next.next`, y así sucesivamente. Exploramos $u_0, u_1, u_2, \dots$ buscando un nodo que pueda proporcionar espacio para `x`. Pueden ocurrir tres casos durante nuestra exploración de espacio (véase la Figura 3.4):

1. Encontramos rápidamente (en $r + 1 \leq b$ pasos) un nodo u_r cuyo bloque no está lleno. En este caso, realizamos r desplazamientos de un elemento de un bloque al siguiente, de modo que el espacio libre en u_r se convierte en un espacio libre en u_0 . Luego podemos insertar `x` en el bloque de u_0 .
2. Rápidamente (en $r + 1 \leq b$ pasos) llegamos al final de la lista de bloques. En este caso, agregamos un nuevo bloque vacío al final de la lista de bloques y procedemos como en el primer caso.
3. Después de b pasos no encontramos ningún bloque que no esté lleno. En este caso, $u_0, \dots, u_{b-1}$ es una secuencia de b bloques que contienen $b + 1$ elementos cada uno. Insertamos un nuevo bloque u_b al final de esta secuencia y distribuimos los $b(b + 1)$ elementos originales de modo que cada bloque de $u_0, \dots, u_b$ contenga exactamente b elementos. Ahora el bloque de u_0 contiene solo b elementos, por lo que tiene espacio para que insertemos `x`.

SEList

```

void add(int i, T x) {
    if (i == n) {
        add(x);
        return;
    }
    Location l; getLocation(i, l);
    Node *u = l.u;
    int r = 0;
    while (r < b && u != &dummy && u->d.size() == b+1) {
        u = u->next;
        r++;
    }
    if (r == b) { // b bloques, cada uno con b+1 elementos
        spread(l.u);
        u = l.u;
    }
    if (u == &dummy) { // se salió del final - agregar nuevo nodo
        u = addBefore(u);
    }
    while (u != l.u) { // trabajar hacia atrás, desplazando elementos
        u->d.add(0, u->prev->d.remove(u->prev->d.size()-1));
        u = u->prev;
    }
    u->d.add(l.j, x);
    n++;
}

```

El tiempo de ejecución de la operación `add(i, x)` depende de cuál de los tres casos anteriores ocurra. Los casos 1 y 2 implican examinar y desplazar elementos a través de como máximo b bloques y toman tiempo $O(b)$. El caso 3 implica llamar al método `spread(u)`, el cual mueve $b(b + 1)$ elementos y toma tiempo $O(b^2)$. Si ignoramos el costo del Caso 3 (el cual contabilizaremos más adelante con amortización), esto significa que el tiempo de ejecución total para localizar i y realizar la inserción de x es $O(b + \min\{i, n - i\}/b)$.



3.3.4 Eliminación de un Elemento

Eliminar un elemento de una `SEList` es similar a agregar un elemento. Primero localizamos el nodo u que contiene el elemento con índice i . Ahora, tenemos que estar preparados para el caso en el que no podemos eliminar un elemento de u sin causar que el bloque de u se vuelva más pequeño que $b - 1$.

De nuevo, sean $u_0, u_1, u_2, \dots$ que denotan $u, u.next, u.next.next$, y así sucesivamente. Examinamos $u_0, u_1, u_2, \dots$ en orden para buscar un nodo del cual podamos tomar prestado un elemento para hacer que el tamaño del bloque de u_0 sea al menos $b - 1$. Hay tres casos a considerar (véase la Figura 3.5):

1. Encontramos rápidamente (en $r + 1 \leq b$ pasos) un nodo cuyo bloque contiene más de $b - 1$ elementos. En este caso, realizamos r desplazamientos de un elemento de un bloque al anterior, de

modo que el elemento extra en u_r se convierte en un elemento extra en u_0 . Luego podemos eliminar el elemento apropiado del bloque de u_0 .

2. Rápidamente (en $r + 1 \leq b$ pasos) llegamos al final de la lista de bloques. En este caso, u_r es el último bloque, y no hay necesidad de que el bloque de u_r contenga al menos $b - 1$ elementos. Por lo tanto, procedemos como se indicó anteriormente, tomando prestado un elemento de u_r para hacer un elemento extra en u_0 . Si esto causa que el bloque de u_r se vuelva vacío, entonces lo eliminamos.
3. Después de b pasos, no encontramos ningún bloque que contenga más de $b - 1$ elementos. En este caso, $u_0, \dots, u_{b-1}$ es una secuencia de b bloques que contienen cada uno $b - 1$ elementos. Reunimos estos $b(b - 1)$ elementos en $u_0, \dots, u_{b-2}$ de modo que cada uno de estos $b - 1$ bloques contenga exactamente b elementos y eliminamos u_{b-1} , que ahora está vacío. Ahora el bloque de u_0 contiene b elementos y podemos entonces eliminar el elemento apropiado de él.

SEList

```
T remove(int i) {
    Location l; getLocation(i, l);
    T y = l.u->d.get(l.j);
    Node *u = l.u;
    int r = 0;
    while (r < b && u != &dummy && u->d.size() == b - 1) {
        u = u->next;
        r++;
    }
    if (r == b) { // se encontraron b bloques, cada uno con b-1 elementos
        gather(l.u);
    }
    u = l.u;
    u->d.remove(l.j);
    while (u->d.size() < b - 1 && u->next != &dummy) {
        u->d.add(u->next->d.remove(0));
        u = u->next;
    }
    if (u->d.size() == 0)
        remove(u);
    n--;
    return y;
}
```

Al igual que la operación $\text{add}(i, x)$, el tiempo de ejecución de la operación $\text{remove}(i)$ es $O(b + \min\{i, n - i\}/b)$ si ignoramos el costo del método $\text{gather}(u)$ que ocurre en el Caso 3.

3.3.5 Análisis Amortizado de Dispersión y Recolección

A continuación, consideramos el costo de los métodos $\text{gather}(u)$ y $\text{spread}(u)$ que pueden ser ejecutados por los métodos $\text{add}(i, x)$ y $\text{remove}(i)$. En aras de la exhaustividad, aquí están:

SEList

```

void spread(Node *u) {
    Node *w = u;
    for (int j = 0; j < b; j++) {
        w = w->next;
    }
    w = addBefore(w);
    while (w != u) {
        while (w->d.size() < b)
            w->d.add(0, w->prev->d.remove(w->prev->d.size()-1));
        w = w->prev;
    }
}

```

SEList

```

void gather(Node *u) {
    Node *w = u;
    for (int j = 0; j < b-1; j++) {
        while (w->d.size() < b)
            w->d.add(w->next->d.remove(0));
        w = w->next;
    }
    remove(w);
}

```

El tiempo de ejecución de cada uno de estos métodos está dominado por los dos bucles anidados. Tanto el bucle interno como el externo se ejecutan como máximo $b + 1$ veces, por lo que el tiempo de ejecución total de cada uno de estos métodos es $O((b + 1)^2) = O(b^2)$. Sin embargo, el siguiente lema muestra que estos métodos se ejecutan como máximo una de cada b llamadas a `add(i, x)` o `remove(i)`.

Lema 3.1. Si se crea una *SEList* vacía y se realiza cualquier secuencia de $m \geq 1$ llamadas a `add(i, x)` y `remove(i)`, entonces el tiempo total empleado durante todas las llamadas a `spread()` y `gather()` es $O(m)$.

Prueba. Utilizaremos el método de análisis amortizado de potencial. Decimos que un nodo u es *frágil* si su bloque no contiene b elementos (es decir, si es el último nodo, o contiene $b - 1$ o $b + 1$ elementos). Cualquier nodo cuyo bloque contiene b elementos es *robusto*. Defina el potencial de una *SEList* como el número de nodos frágiles que contiene. Consideraremos solo la operación `add(i, x)` y su relación con el número de llamadas a `spread(u)`. El análisis de `remove(i)` y `gather(u)` es idéntico.

Observe que, si el Caso 1 ocurre durante `add(i, x)`, entonces solo un nodo, u_i , cambia el tamaño de su bloque. Por lo tanto, como máximo un nodo, a saber, u_i , pasa de ser robusto a frágil. Si ocurre el Caso 2, entonces se crea un nuevo nodo, y este nodo es frágil, pero ningún otro nodo cambia de tamaño, por lo que el número de nodos frágiles aumenta en uno. Así, en el Caso 1 o en el Caso 2 el potencial de la *SEList* aumenta como máximo en uno.

Finalmente, si ocurre el Caso 3, es porque $u_0, \dots, u_{b-1}$ son todos bloques frágiles. Luego `spread(u0)` es llamado y estos b nodos frágiles son reemplazados por $b + 1$ nodos robustos. Finalmente, se agrega x al

bloque de u_0 , haciendo que u_0 sea frágil. En total, el potencial disminuye en $b - 1$.

En resumen, el potencial comienza en 0 (no hay nodos en la lista). Cada vez que ocurre el Caso 1 o el Caso 2, el potencial aumenta como máximo en 1. Cada vez que ocurre el Caso 3, el potencial disminuye en $b - 1$. El potencial (que cuenta el número de nodos frágiles) nunca es menor que 0. Concluimos que, por cada ocurrencia del Caso 3, hay al menos $b - 1$ ocurrencias del Caso 1 o del Caso 2. Así, por cada llamada a `spread(u)` hay al menos b llamadas a `add(i, x)`. Esto completa la prueba. $\square$

3.3.6 Resumen

El siguiente teorema resume el rendimiento de la estructura de datos `SEList`:

Teorema 3.3. Una `SEList` implementa la interfaz `List`. Ignorando el costo de las llamadas a `spread(u)` y `gather(u)`, una `SEList` con tamaño de bloque b soporta las operaciones

- `get(i)` y `set(i, x)` en tiempo $O(1 + \min\{i, n - i\}/b)$ por operación; y
- `add(i, x)` y `remove(i)` en tiempo $O(b + \min\{i, n - i\}/b)$ por operación.

Además, comenzando con una `SEList` vacía, cualquier secuencia de m operaciones `add(i, x)` y `remove(i)` resulta en un total de $O(bm)$ tiempo empleado durante todas las llamadas a `spread(u)` y `gather(u)`.

El espacio (medido en palabras)³ utilizado por una `SEList` que almacena n elementos es $n + O(b + n/b)$.

La `SEList` es un compromiso entre un `ArrayStack` y una `DLList`, donde la mezcla relativa de estas dos estructuras depende del tamaño de bloque b . En el extremo $b = 2$, cada nodo de la `SEList` almacena como máximo tres valores, lo que no es muy diferente de una `DLList`. En el otro extremo, $b > n$, todos los elementos se almacenan en un solo arreglo, al igual que en un `ArrayStack`. Entre estos dos extremos se encuentra un compromiso entre el tiempo que toma agregar o eliminar un elemento de la lista y el tiempo que toma localizar un elemento particular de la lista.

3.4 Discusión y Ejercicios

Tanto las listas simplemente enlazadas como las doblemente enlazadas son técnicas establecidas, que se han utilizado en programas durante más de 40 años. Son discutidas, por ejemplo, por Knuth [46, Secciones 2.2.3-2.2.5]. Incluso la estructura de datos `SEList` parece ser un ejercicio bien conocido de estructuras de datos. A la `SEList` a veces se la conoce como una lista enlazada desenrollada (*unrolled linked list*) [67].

Otra forma de ahorrar espacio en una lista doblemente enlazada es utilizar las llamadas listas XOR. En una lista XOR, cada nodo, u , contiene solo un puntero, llamado `u.nextprev`, que contiene el XOR bit a bit de `u.prev` y `u.next`. La lista en sí necesita almacenar dos punteros, uno al nodo ficticio (`dummy`) y otro a `dummy.next` (el primer nodo, o `dummy` si la lista está vacía). Esta técnica utiliza el hecho de que, si tenemos punteros a u y `u.prev`, entonces podemos extraer `u.next` usando la fórmula

$$u.next = u.prev \hat{=} u.nextprev.$$

(Aquí $\hat{=}$ calcula el XOR bit a bit de sus dos argumentos.) Esta técnica complica un poco el código y no es posible en algunos lenguajes, como Java y Python, que tienen recolección de basura, pero proporciona una implementación de lista doblemente enlazada que requiere solo un puntero por nodo. Véase el artículo de revista de Sinha [68] para una discusión detallada de las listas XOR.

Ejercicio 3.1. ¿Por qué no es posible usar un nodo ficticio (*dummy*) en una *SLList* para evitar todos los casos especiales que ocurren en las operaciones *push(x)*, *pop()*, *add(x)* y *remove()*?

Ejercicio 3.2. Diseñe e implemente un método *SLList*, *secondLast()*, que devuelva el penúltimo elemento de una *SLList*. Haga esto sin usar la variable miembro, *n*, que lleva el registro del tamaño de la lista.

Ejercicio 3.3. Implemente las operaciones de *List* *get(i)*, *set(i,x)*, *add(i,x)* y *remove(i)* en una *SLList*. Cada una de estas operaciones debe ejecutarse en tiempo $O(1 + i)$.

Ejercicio 3.4. Diseñe e implemente un método *SLList*, *reverse()*, que invierta el orden de los elementos en una *SLList*. Este método debe ejecutarse en tiempo $O(n)$, no debe usar recursión, no debe usar estructuras de datos secundarias y no debe crear nuevos nodos.

Ejercicio 3.5. Diseñe e implemente métodos *SLList* y *DLList* llamados *checkSize()*. Estos métodos recorren la lista y cuentan el número de nodos para ver si esto coincide con el valor, *n*, almacenado en la lista. Estos métodos no devuelven nada, pero lanzan una excepción si el tamaño que calculan no coincide con el valor de *n*.

Ejercicio 3.6. Intente recrear el código para la operación *addBefore(w)* que crea un nodo, *u*, y lo agrega en una *DLList* justo antes del nodo *w*. No consulte este capítulo. Incluso si su código no coincide exactamente con el código dado en este libro, aún puede ser correcto. Pruébalo y vea si funciona.

Los siguientes ejercicios implican realizar manipulaciones en *DLLists*. Debe completarlos sin asignar nuevos nodos ni arreglos temporales. Todos pueden hacerse solo cambiando los valores *prev* y *next* de los nodos existentes.

Ejercicio 3.7. Escriba un método de *DLList* *isPalindrome()* que devuelva *true* si la lista es un palíndromo, es decir, el elemento en la posición *i* es igual al elemento en la posición $n - i - 1$ para todo $i \in \{0, \dots, n - 1\}$. Su código debe ejecutarse en tiempo $O(n)$.

Ejercicio 3.8. Implemente un método *rotate(r)* que "rote" una *DLList* de modo que el elemento de lista *i* se convierta en el elemento de lista $(i + r) \bmod n$. Este método debe ejecutarse en tiempo $O(1 + \min\{r, n - r\})$ y no debe modificar ningún nodo en la lista.

Ejercicio 3.9. Escriba un método, *truncate(i)*, que trunca una *DLList* en la posición *i*. Después de ejecutar este método, el tamaño de la lista será *i* y solo debería contener los elementos en los índices $0, \dots, i - 1$. El valor de retorno es otra *DLList* que contiene los elementos en los índices $i, \dots, n - 1$. Este método debe ejecutarse en tiempo $O(\min\{i, n - i\})$.

Ejercicio 3.10. Escriba un método de *DLList*, *absorb(l2)*, que toma como argumento una *DLList*, *l2*, la vacía y agrega su contenido, en orden, al receptor. Por ejemplo, si *l1* contiene *a*, *b*, *c* y *l2* contiene *d*, *e*, *f*, entonces después de llamar a *l1.absorb(l2)*, *l1* contendrá *a*, *b*, *c*, *d*, *e*, *f* y *l2* estará vacía.

Ejercicio 3.11. Escriba un método *deal()* que elimina todos los elementos con índices impares de una *DLList* y devuelve una *DLList* que contiene estos elementos. Por ejemplo, si *l1* contiene los elementos *a*, *b*, *c*, *d*, *e*, *f*, entonces después de llamar a *l1.deal()*, *l1* debería contener *a*, *c*, *e* y una lista que contiene *b*, *d*, *f* debería ser devuelta.

Ejercicio 3.12. Escriba un método, *reverse()*, que invierta el orden de los elementos en una *DLList*.

Ejercicio 3.13. Este ejercicio lo guía a través de una implementación del algoritmo de ordenamiento por mezcla (*merge-sort*) para ordenar una `DLList`, como se discute en la Sección 11.1.1.

1. Escriba un método de `DLList` llamado `takeFirst(12)`. Este método toma el primer nodo de `12` y lo agrega a la lista receptora. Esto es equivalente a `add(size(), 12.remove(0))`, excepto que no debería crear un nuevo nodo.
2. Escriba un método estático de `DLList`, `merge(11, 12)`, que tome dos listas ordenadas `11` y `12`, las fusione, y devuelva una nueva lista ordenada que contenga el resultado. Esto hace que `11` y `12` se vacíen en el proceso.

Por ejemplo, si `11` contiene `a, c, d` y `12` contiene `b, e, f`, entonces este método devuelve una nueva lista que contiene `a, b, c, d, e, f`.

3. Escriba un método de `DLList` `sort()` que ordene los elementos contenidos en la lista utilizando el algoritmo de ordenamiento por mezcla (*merge sort*). Este algoritmo recursivo funciona de la siguiente manera:
 - (a) Si la lista contiene 0 o 1 elementos, entonces no hay nada que hacer. De lo contrario (b) Utilizando el método `truncate(size()/2)`, divida la lista en dos listas de longitud aproximadamente igual, `11` y `12`;
 - (c) Ordene recursivamente `11`;
 - (d) Ordene recursivamente `12`; y, finalmente,
 - (e) Fusione `11` y `12` en una sola lista ordenada.

Los siguientes ejercicios son más avanzados y requieren una comprensión clara de lo que sucede con el valor mínimo almacenado en una Pila (`Stack`) o Cola (`Queue`) a medida que se agregan y eliminan elementos.

Ejercicio 3.14. Diseñe e implemente una estructura de datos `MinStack` que pueda almacenar elementos comparables y soporte las operaciones de pila `push(x)`, `pop()` y `size()`, así como la operación `min()`, que devuelve el valor mínimo actualmente almacenado en la estructura de datos. Todas las operaciones deben ejecutarse en tiempo constante.

Ejercicio 3.15. Diseñe e implemente una estructura de datos `MinQueue` que pueda almacenar elementos comparables y soporte las operaciones de cola `add(x)`, `remove()` y `size()`, así como la operación `min()`, que devuelve el valor mínimo actualmente almacenado en la estructura de datos. Todas las operaciones deben ejecutarse en tiempo constante amortizado.

Ejercicio 3.16. Diseñe e implemente una estructura de datos `MinDeque` que pueda almacenar elementos comparables y soporte todas las operaciones de deque `addFirst(x)`, `addLast(x)`, `removeFirst()`, `removeLast()` y `size()`, y la operación `min()`, que devuelve el valor mínimo actualmente almacenado en la estructura de datos. Todas las operaciones deben ejecutarse en tiempo constante amortizado.

Los siguientes ejercicios están diseñados para poner a prueba la comprensión del lector sobre la implementación y el análisis de la `SEList` eficiente en espacio:

Ejercicio 3.17. Demuestre que, si se utiliza una `SEList` como una Pila (Pila: de modo que las únicas operaciones sobre la `SEList` sean `push(x) = add(size(), x)` y `pop() = remove(size() - 1)`), entonces estas operaciones se ejecutan en tiempo constante amortizado, independientemente del valor de `b`.

Ejercicio 3.18. Diseñe e implemente una versión de una `SEList` que soporte todas las operaciones de Deque en tiempo constante amortizado por operación, independientemente del valor de `b`.

Ejercicio 3.19. Explique cómo usar el operador exclusivo bit a bit (XOR), `^`, para intercambiar los valores de dos variables `int` sin utilizar una tercera variable.